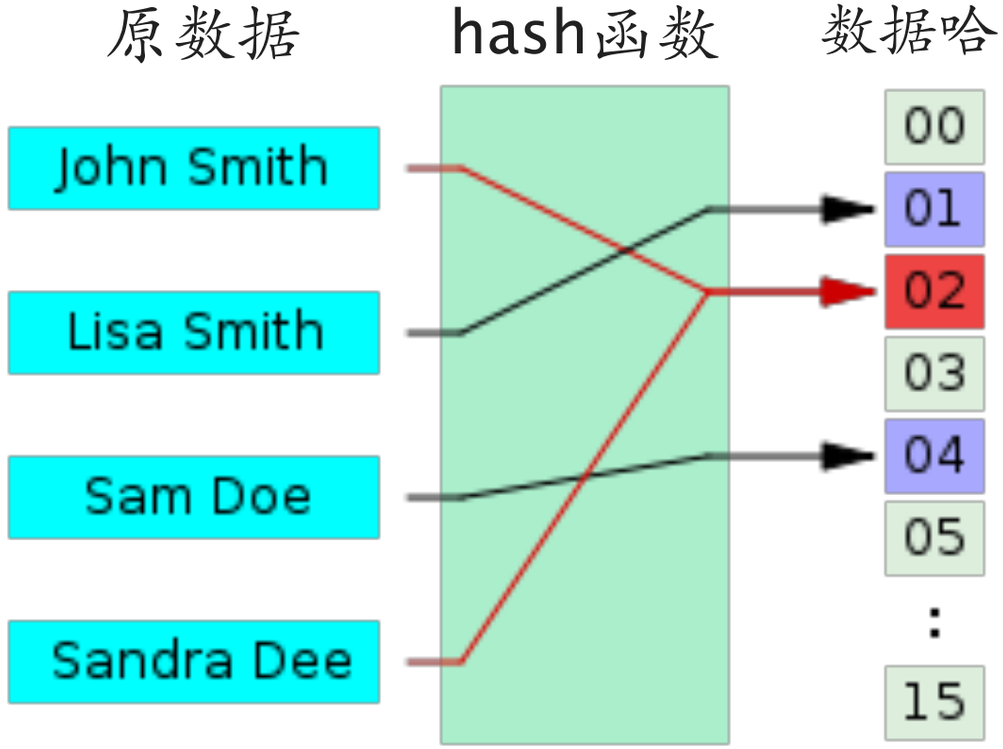
云存储中的数据完整性审计

——由中心式到分布式

阎 萌
yanm@nbjl.nankai.edu.cn

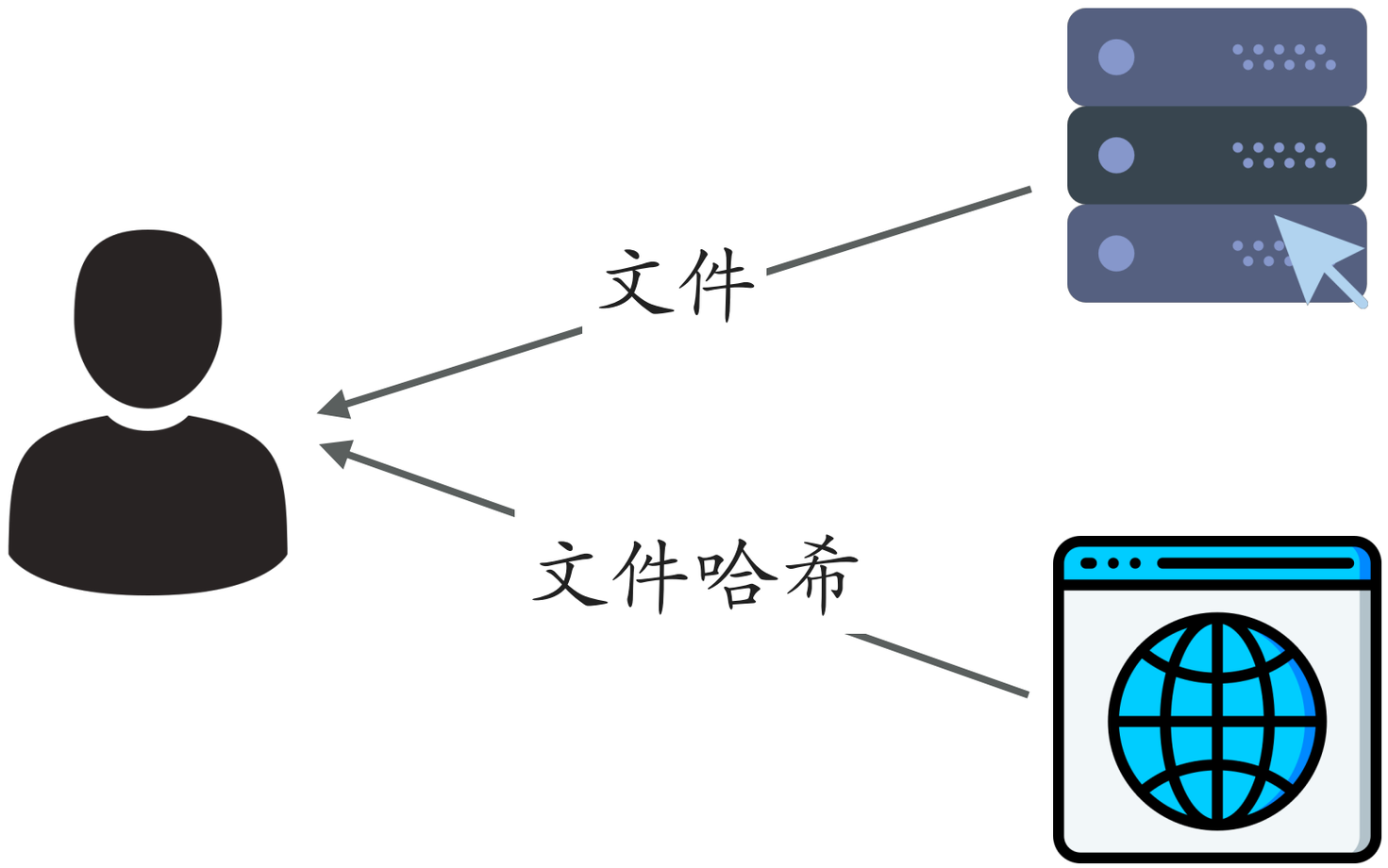
传统完整性验证：HASH (1/3)

- hash函数



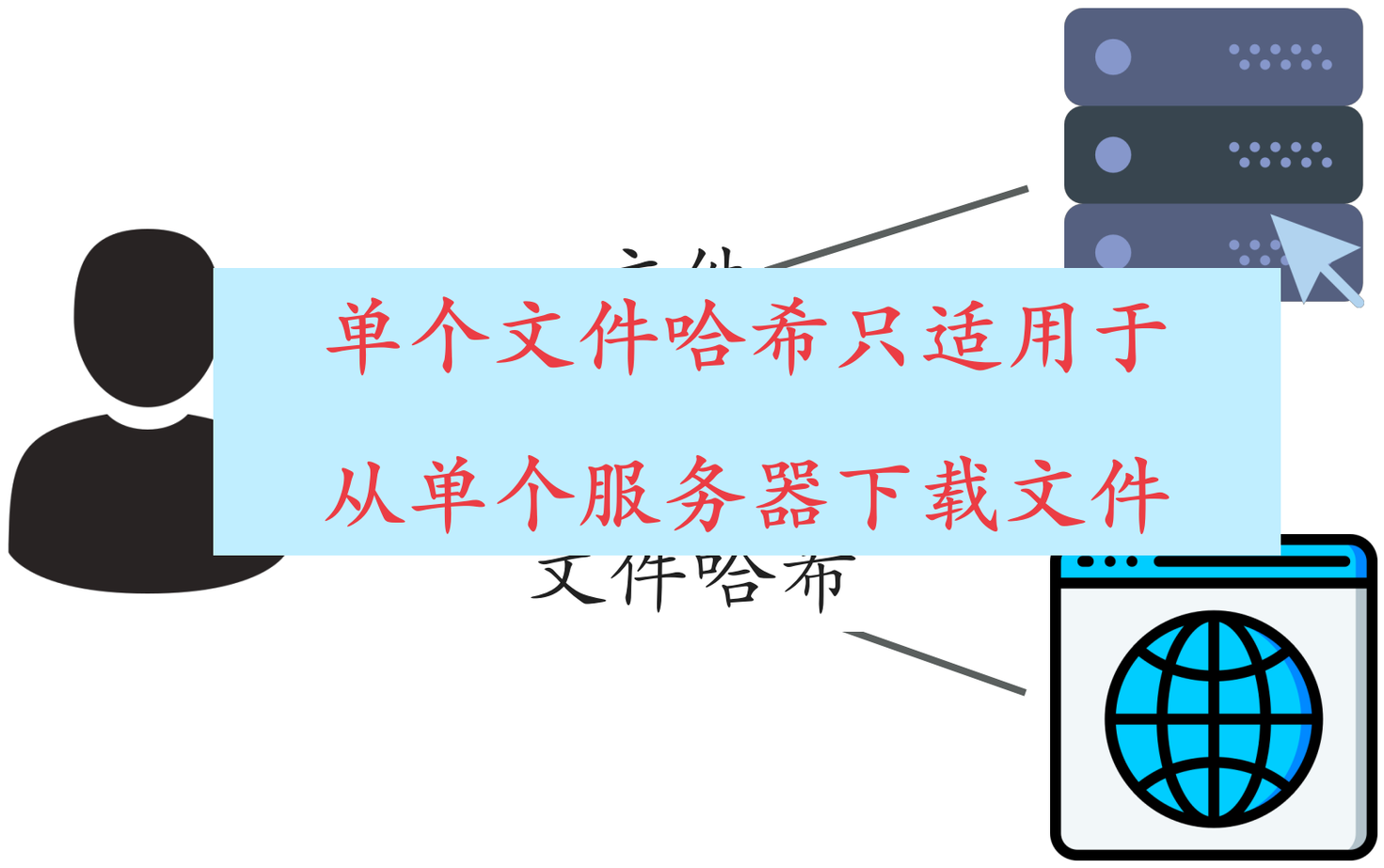
传统完整性验证：HASH (2/3)

- 利用文件哈希验证文件完整性



传统完整性验证：HASH (2/3)

- 利用文件哈希验证文件完整性

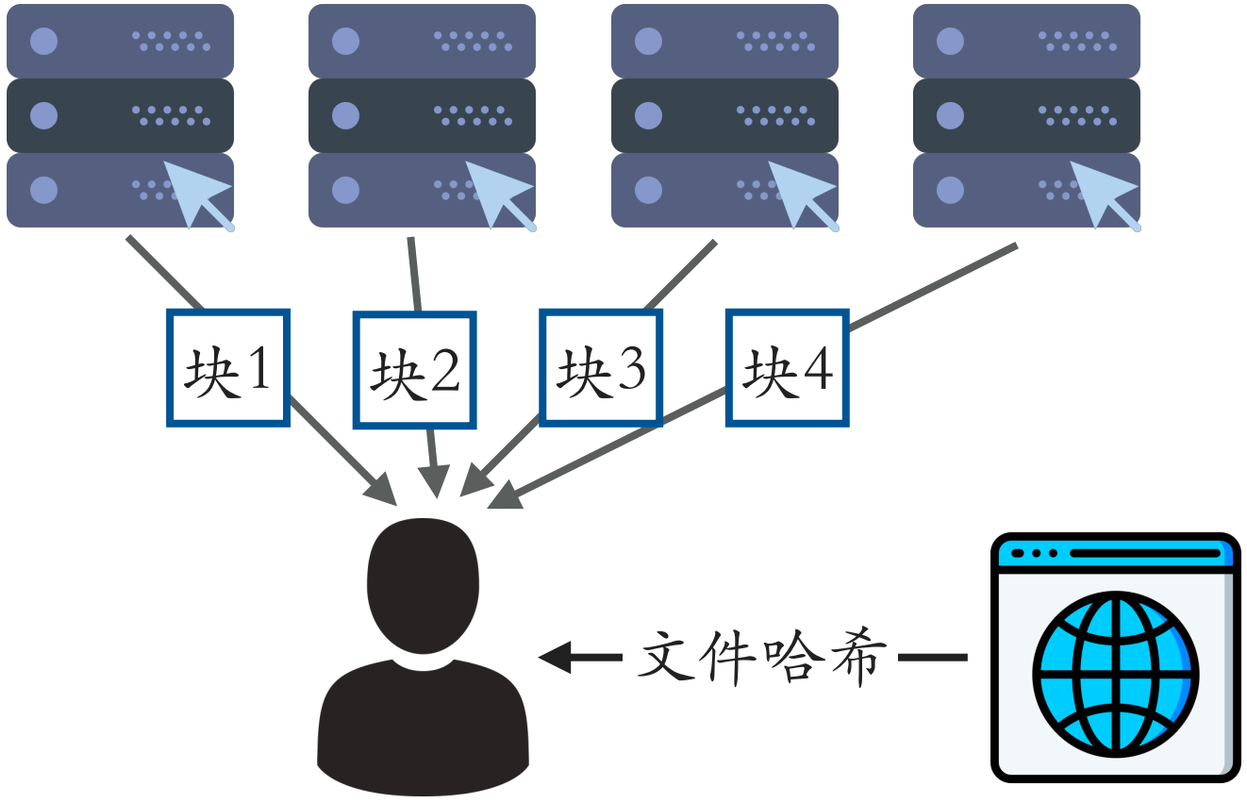


单个文件哈希只适用于

从单个服务器下载文件

传统完整性验证：HASH (3/3)

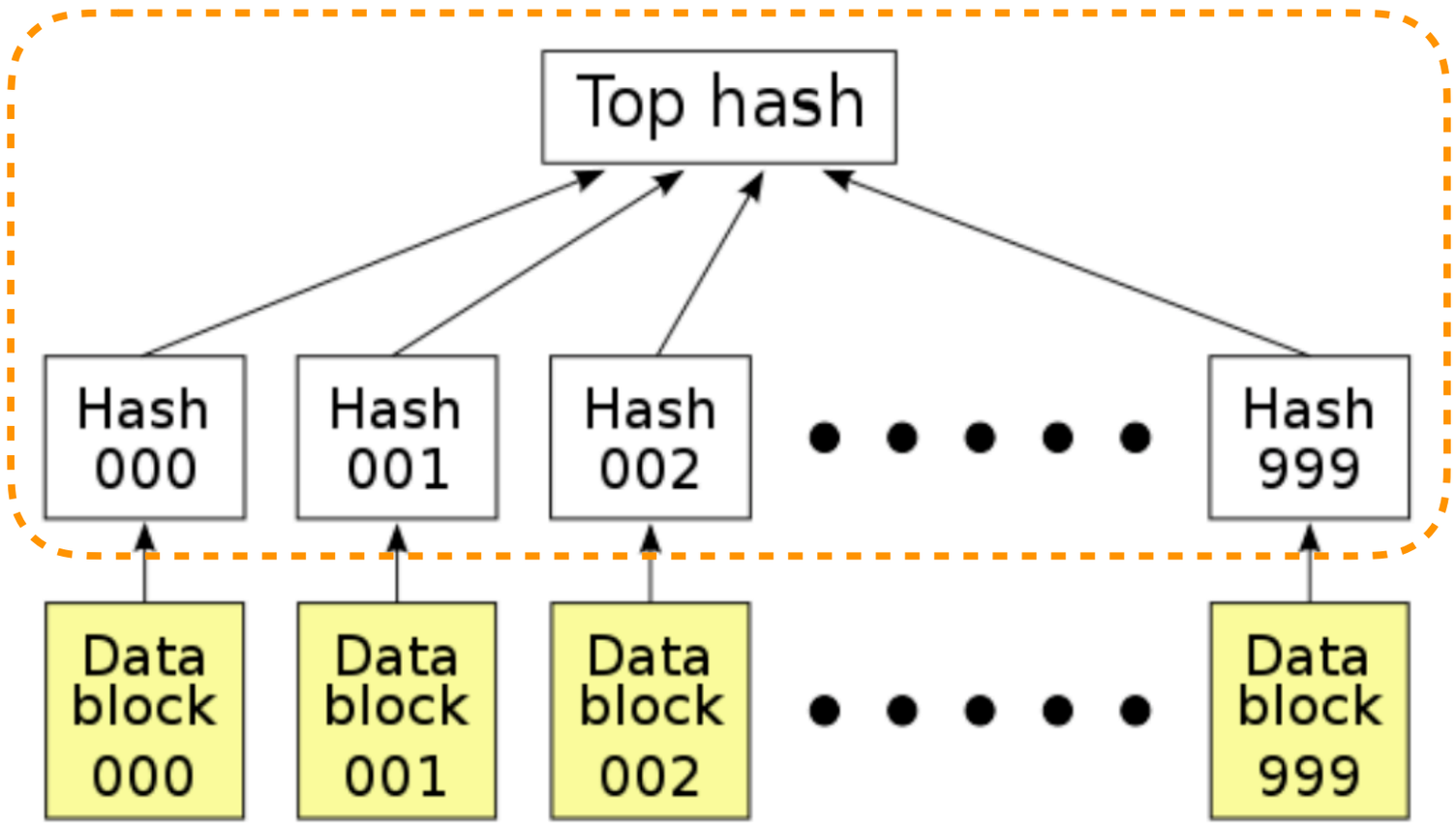
- 若文件分布式存储，则存在错块时不得不反复下载，直到下到的块全是对的。



传统完整性验证：HASH LIST (1/3)

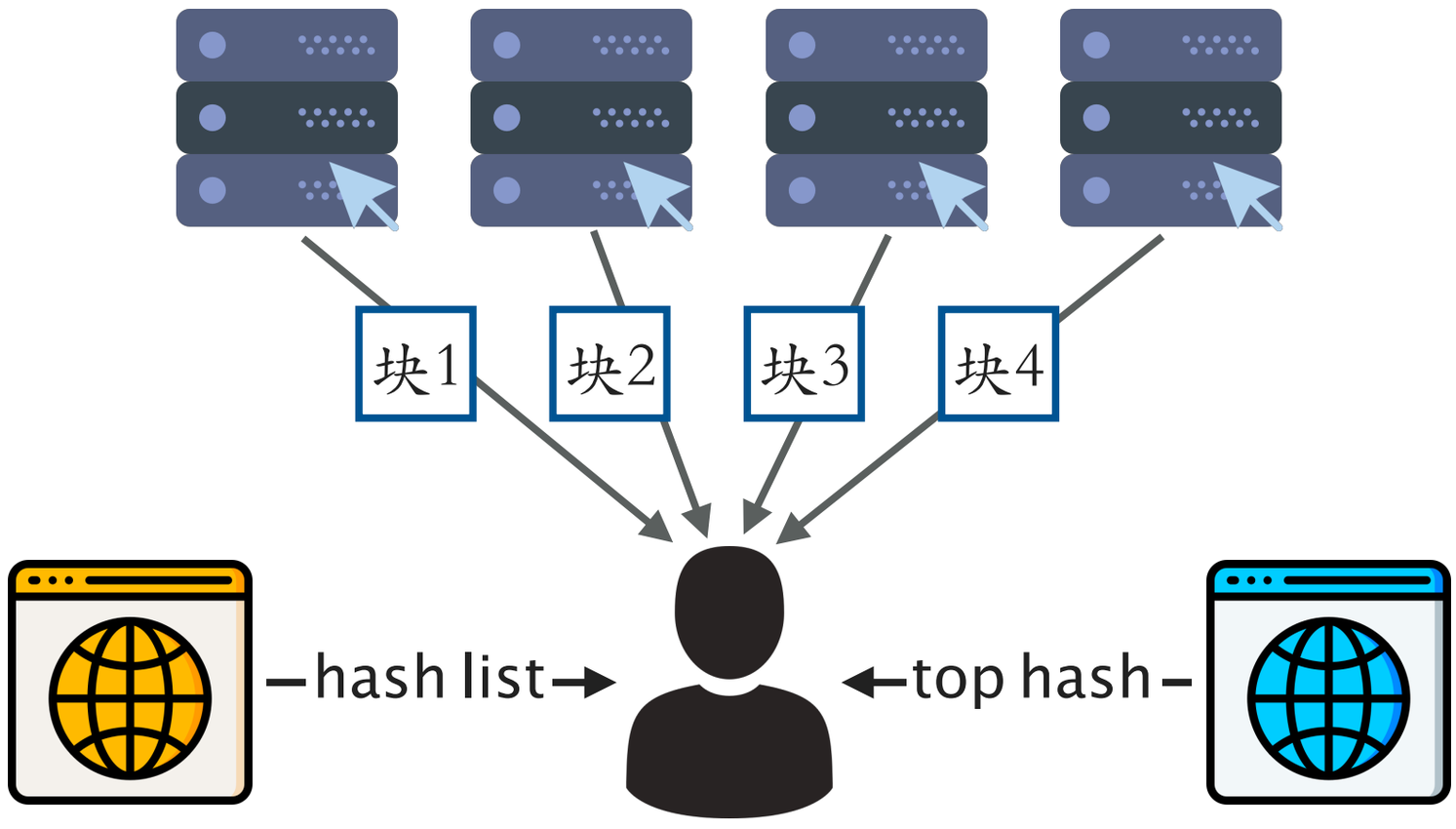
NBJL

- hash list + top hash



传统完整性验证：HASH LIST (2/3)

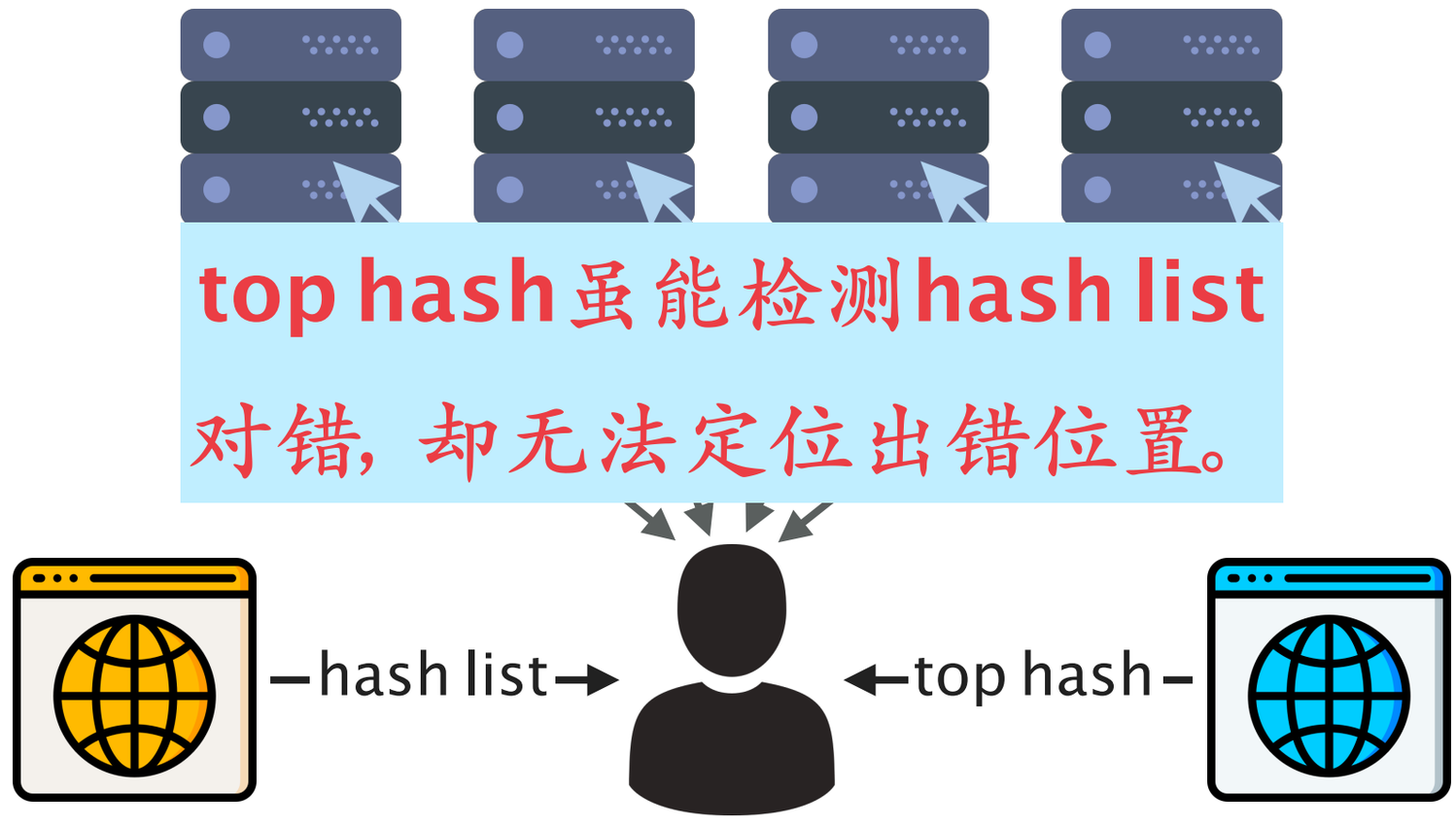
- 利用hash list在分布式网络中验证文件块完整性



传统完整性验证：HASH LIST (2/3)

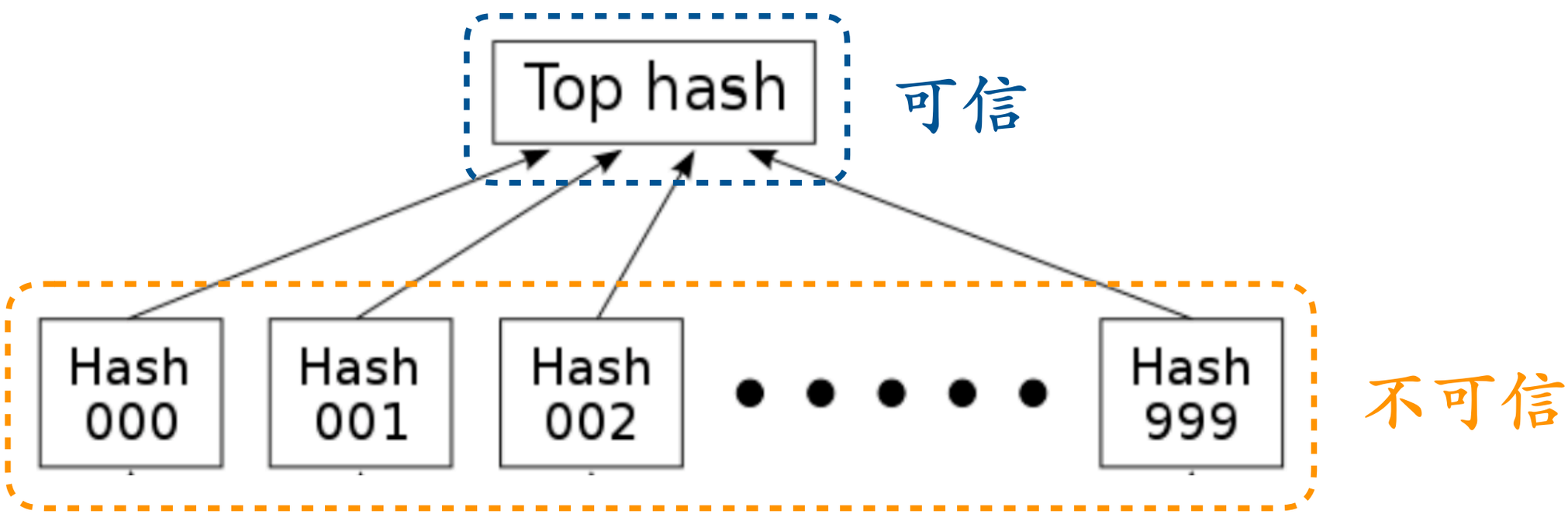
NBJL

- 利用hash list在分布式网络中验证文件块完整性



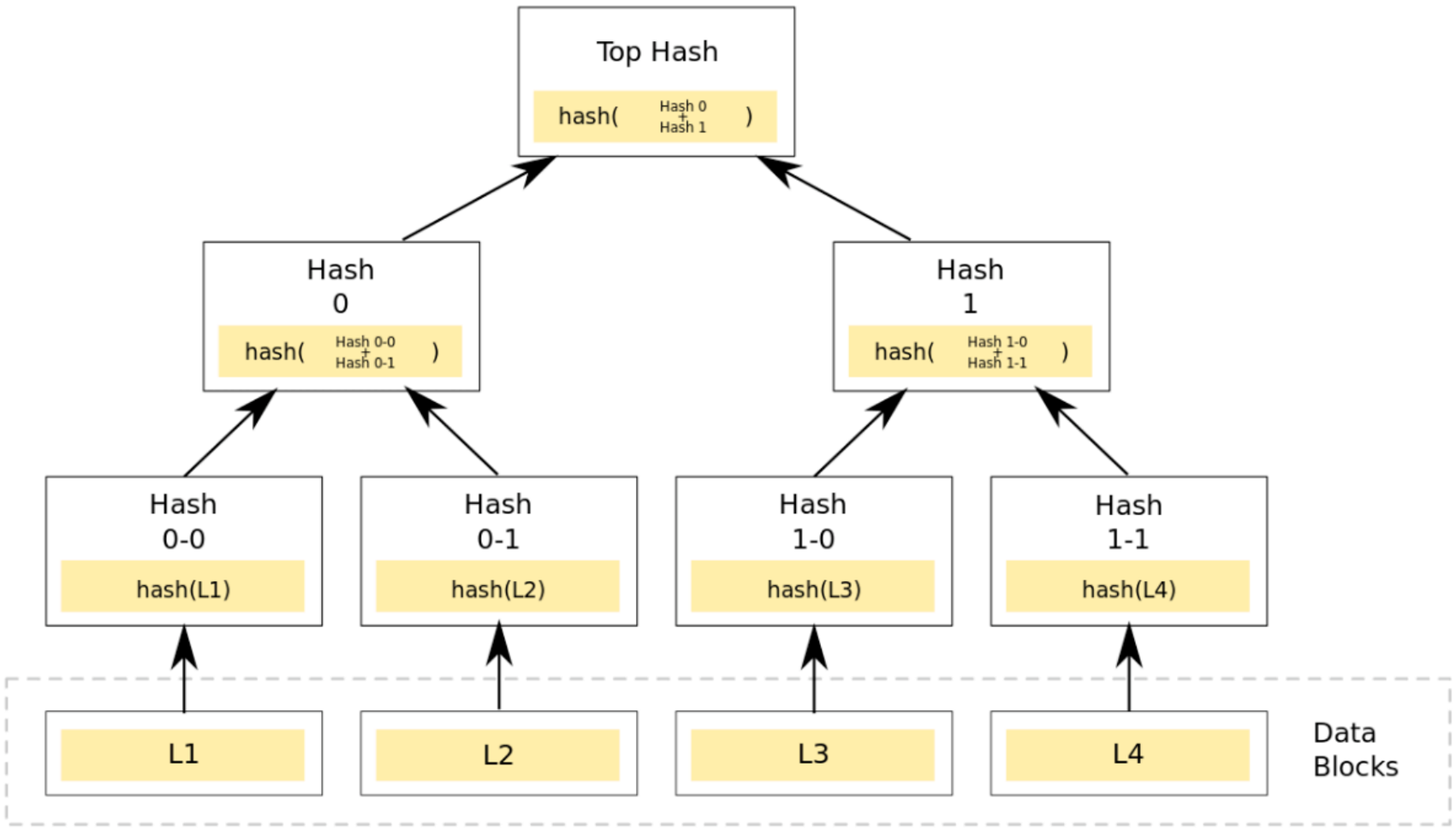
传统完整性验证：HASH LIST (3/3)

- 若hash list错误，只能从其它数据源下载新的hash list并再次验证。



传统完整性验证：MERKLE TREE (1/3)

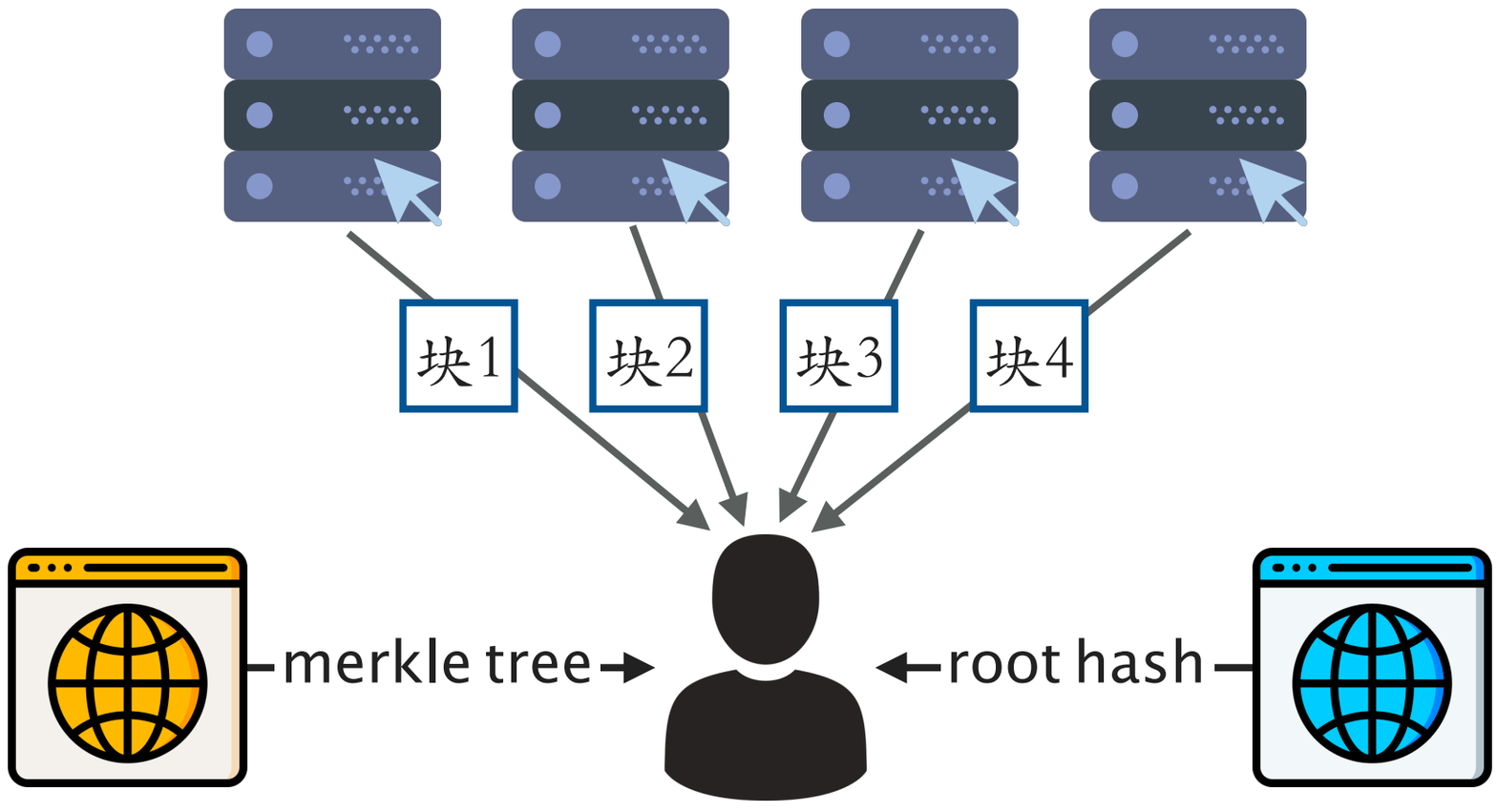
- Merkle Tree



传统完整性验证：MERKLE TREE (2/3)

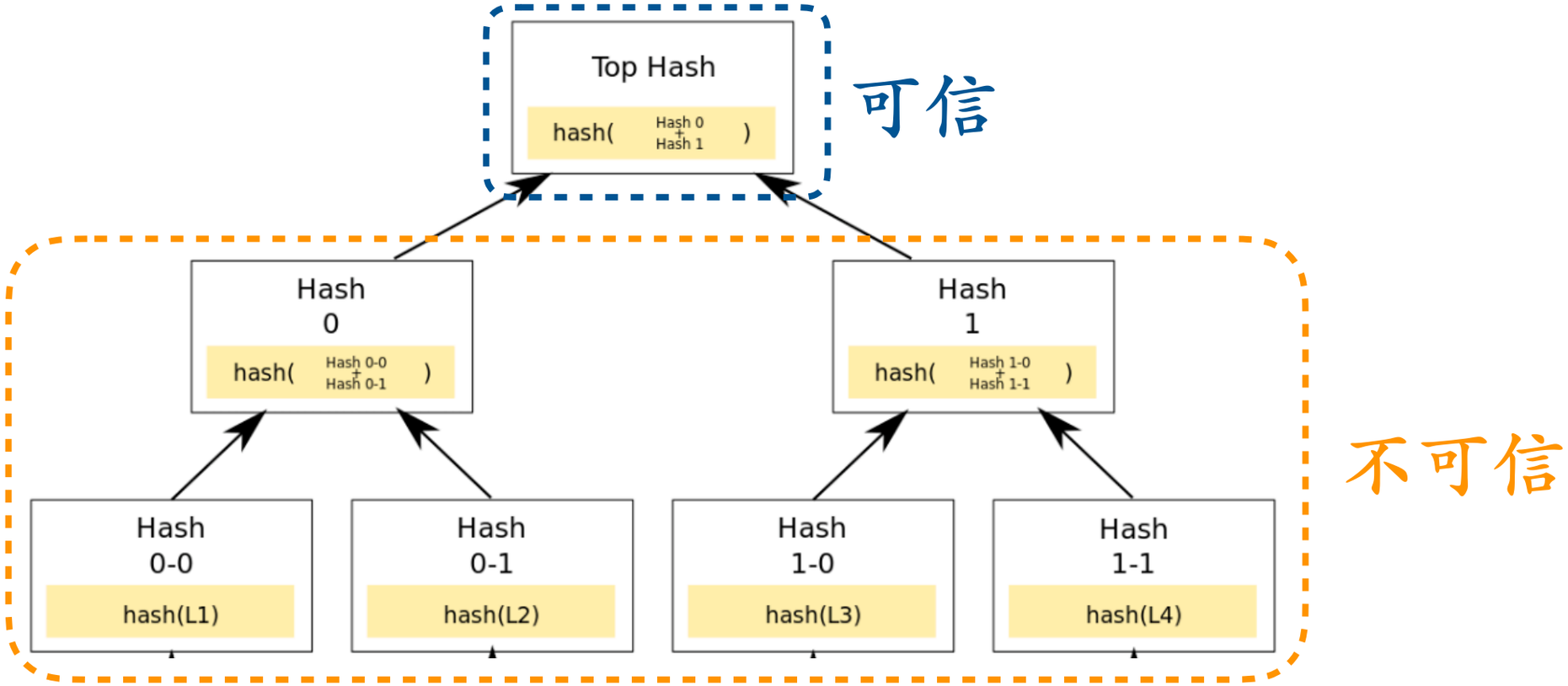
NBJL

- 利用merkle tree在分布式网络中验证文件块完整性



传统完整性验证：MERKLE TREE (3/3)

- 与hash list相比更为高效：可以验证一半的list是否正确。



传统的完整性验证方法

· hash·

· hash list·

· merkle tree·

在不受信的云环境中我们有新的需求

云上的数据时刻保存完好

云存储中的数据完整性

① 时刻

一个直观的方法：一直做验证

云存储中的数据完整性

① 时刻

一个直观的方法：一直做验证 ✗ 浪费资源！

浪费用户时间！

云存储中的数据完整性

① 时刻

一个高效的方法：随机地进行验证

云存储中的数据完整性

② 完好

- 利用传统hash方法验证完整性

云存储中的数据完整性

② 完好

- 利用传统hash方法验证完整性 ✗

需要下载完整文件，繁琐！

云存储中的数据完整性

② 完好

- 无需下载文件便能够验证完整性

云存储中的数据完整性

③ 不可抵赖性

- 用户不能完全相信云的存储服务
- 云不能完全相信用户执行的验证结果

云存储中的数据完整性

③ 不可抵赖性

- 用户不能完全相信云的存储服务
- 云不能完全相信用户执行的验证结果

云存储的数据完整性审计需求

随机发起验证

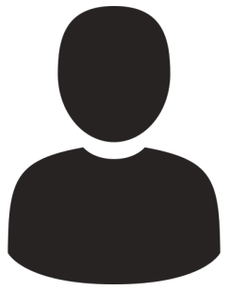
无需下载文件便能验证其完整性

满足不可抵赖性

中心式的审计模型(1/5)

NBJL

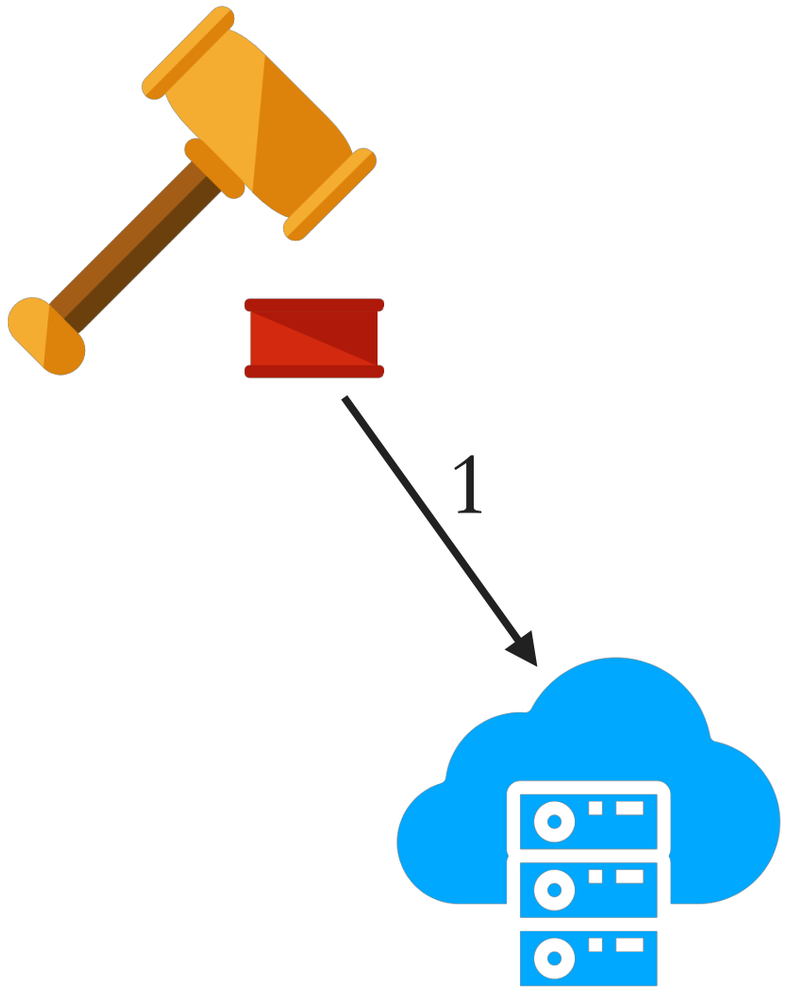
- 三个参与者
  - 用户
  - 可信审计者
  - 云



中心式的审计模型(2/5)

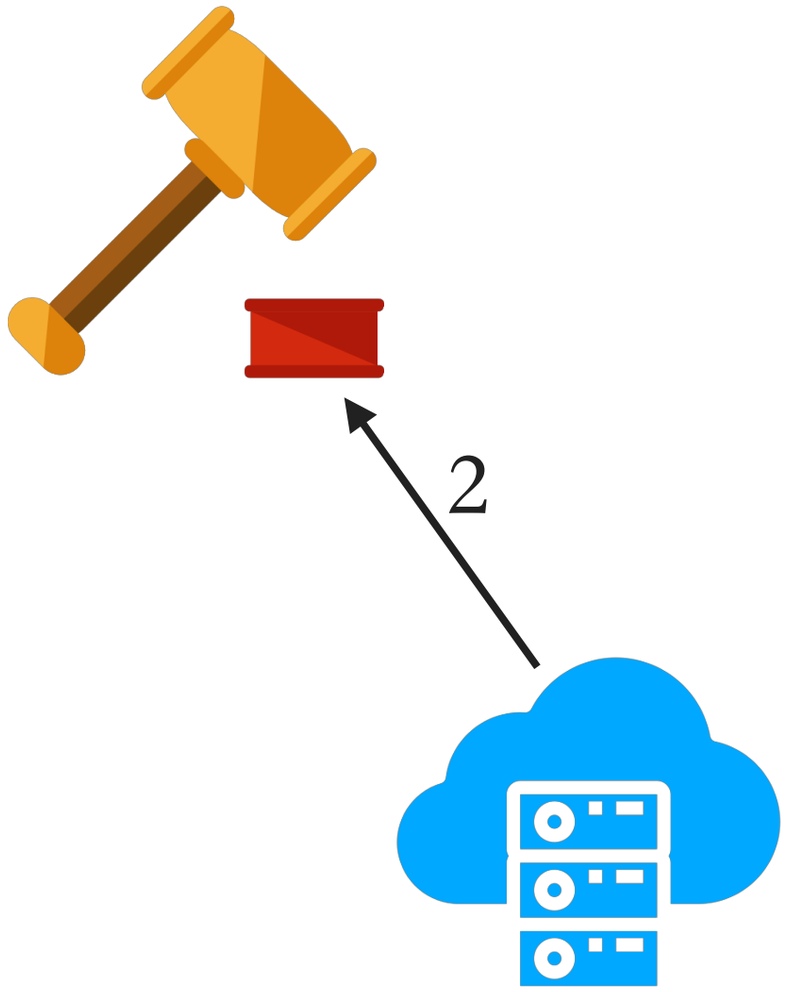
NBJL

1. 审计者发起挑战
   - 时间随机
   - 被选中的文件随机



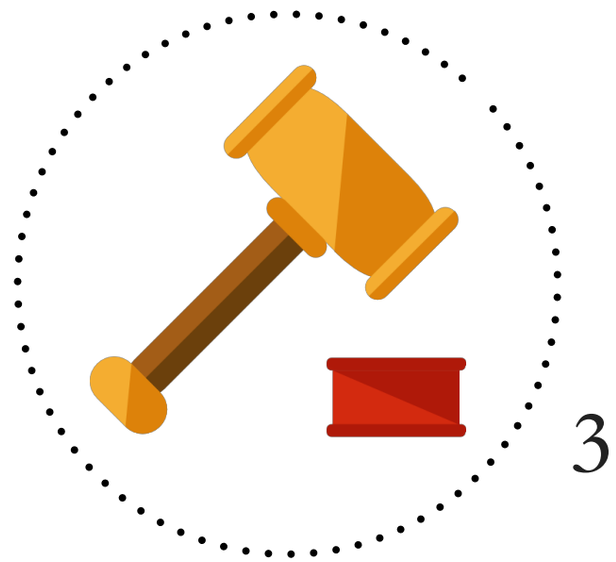
中心式的审计模型(3/5)

2. 云进行响应
   - 证明文件完整性
   - 审计者无需下载文件



中心式的审计模型(4/5)

3. 审计者进行验证

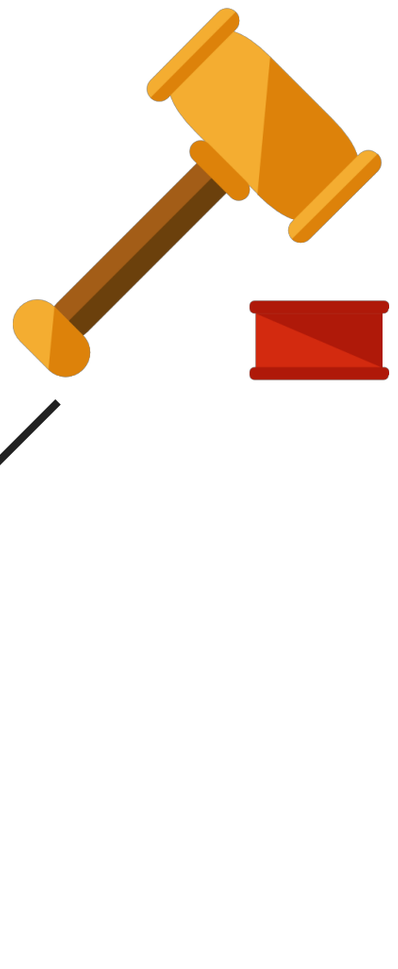


3

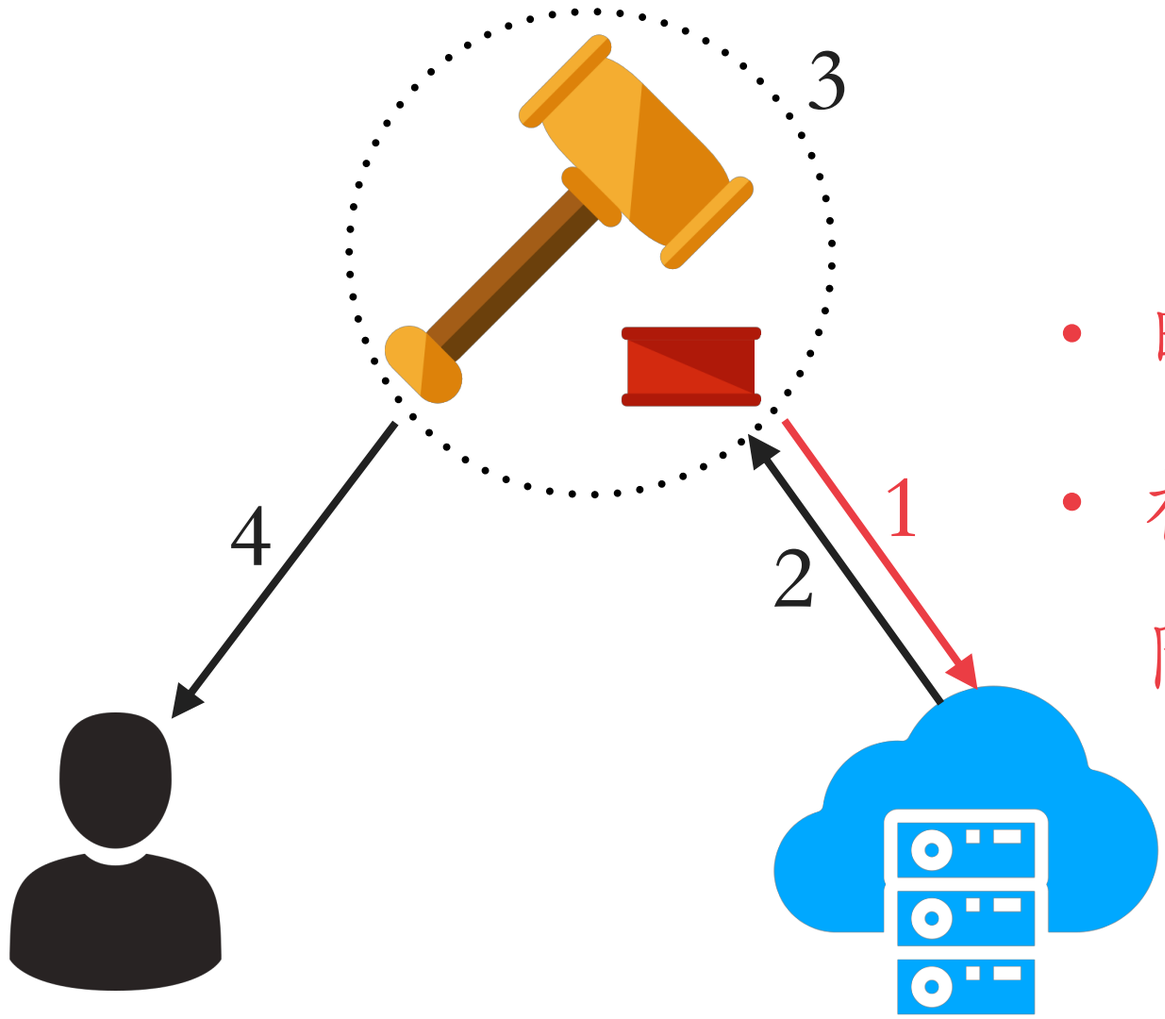
中心式的审计模型(5/5)

NBJL

4. 审计者将结果告知用户

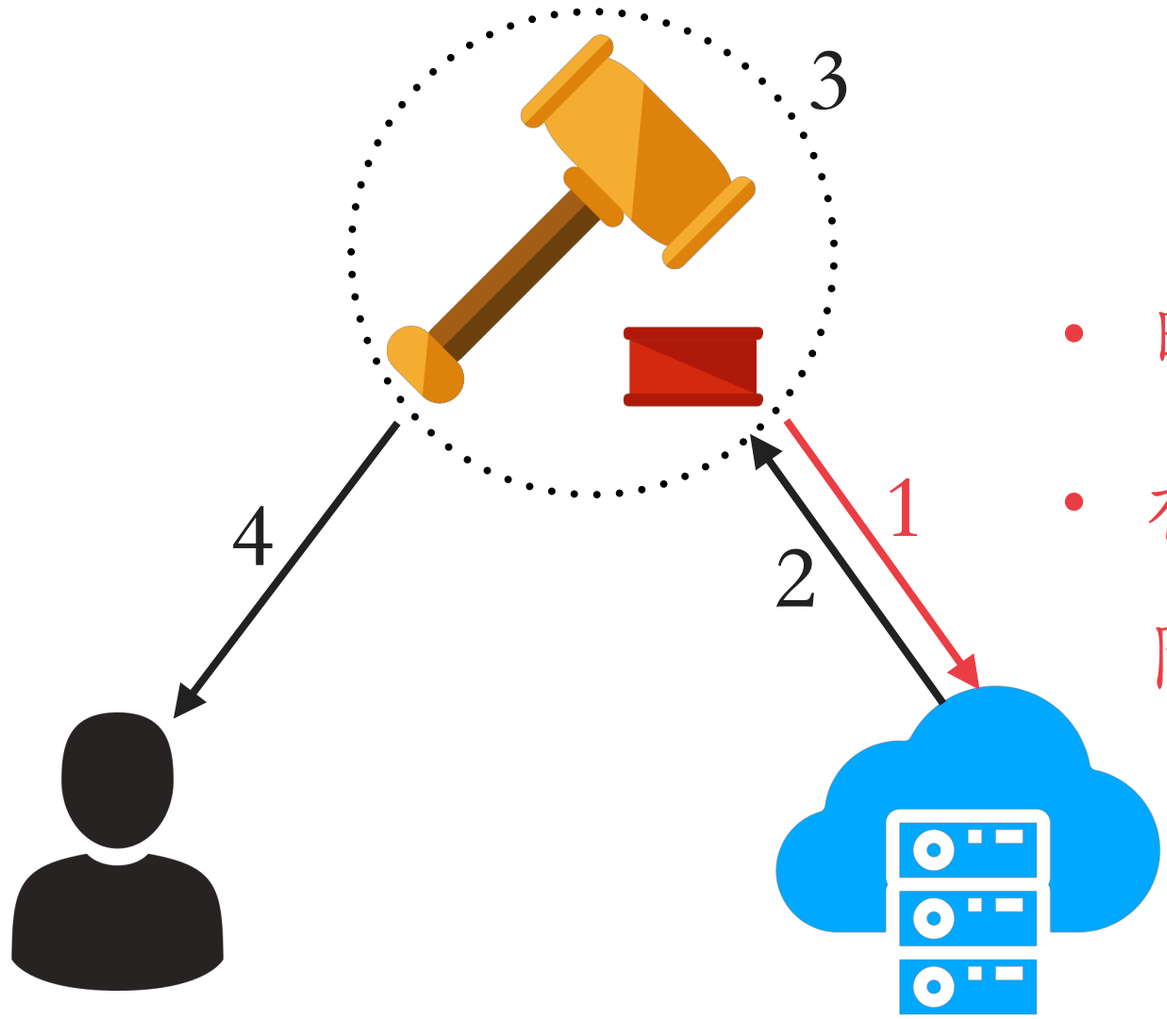


中心式审计模型存在的问题



- 时间随机?
- 被选中的文件随机?

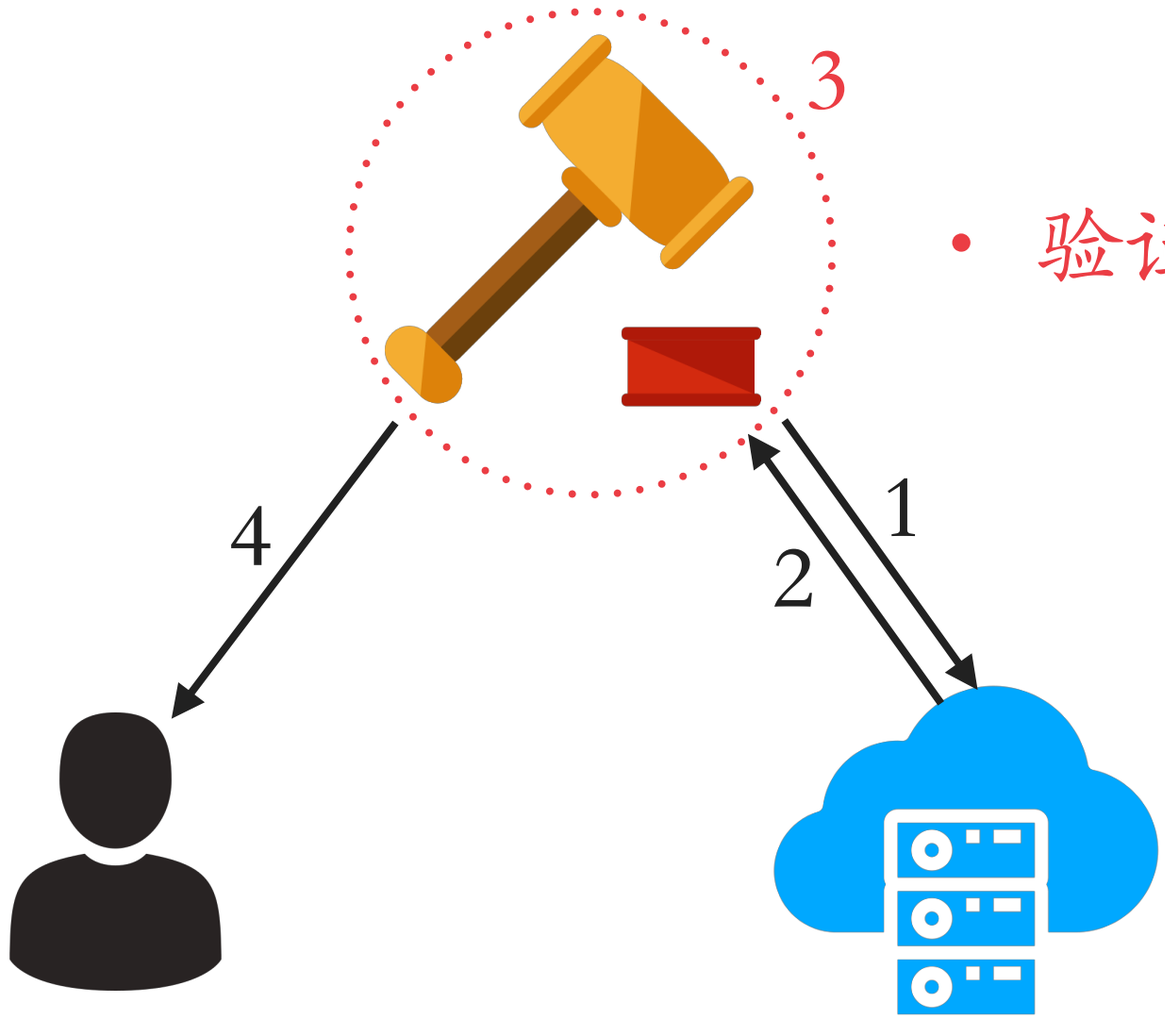
中心式审计模型存在的问题



- 时间随机?
- 被选中的文件随机?

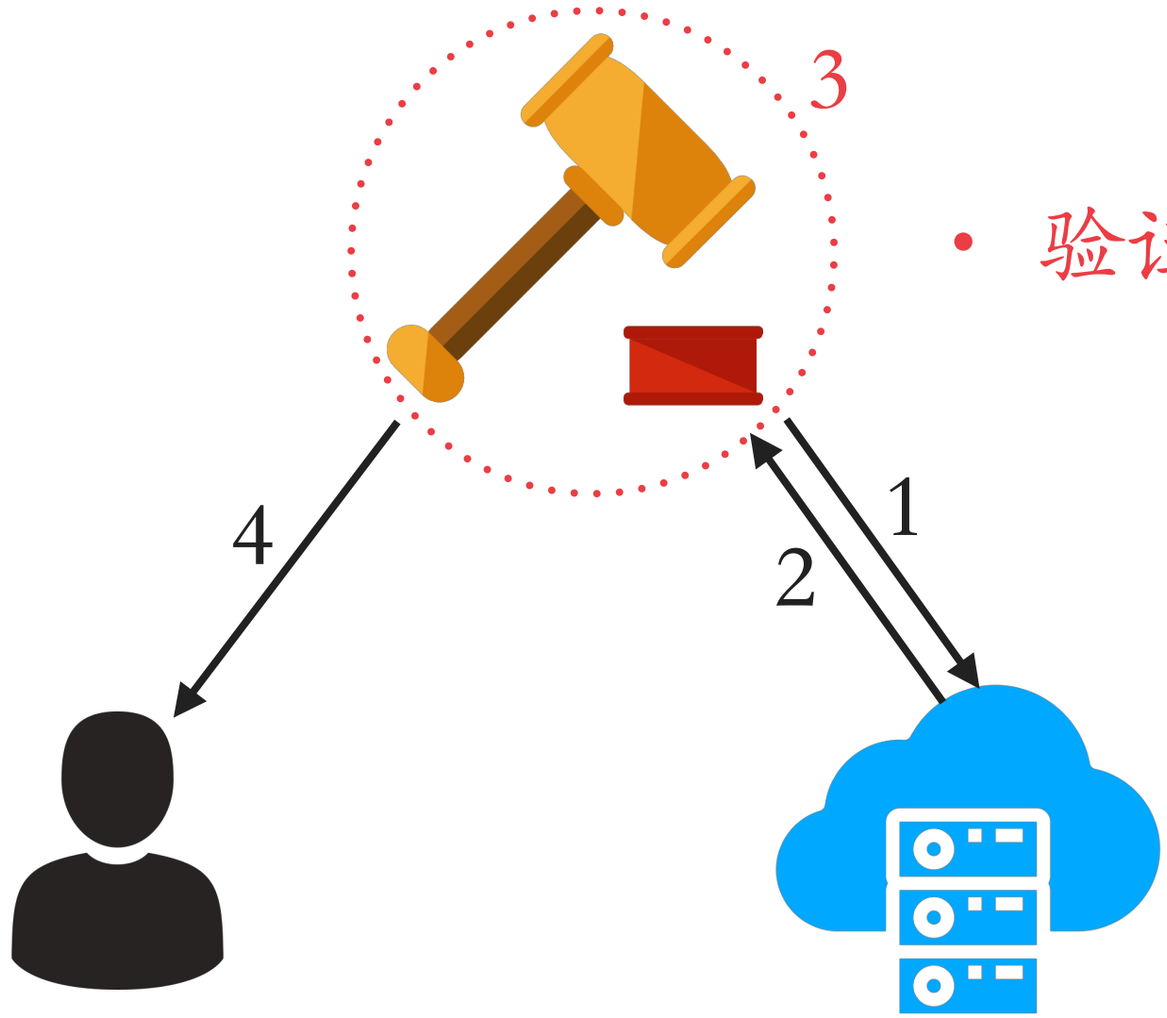
无法验证!

中心式审计模型存在的问题



- 验证过程公正?

中心式审计模型存在的问题

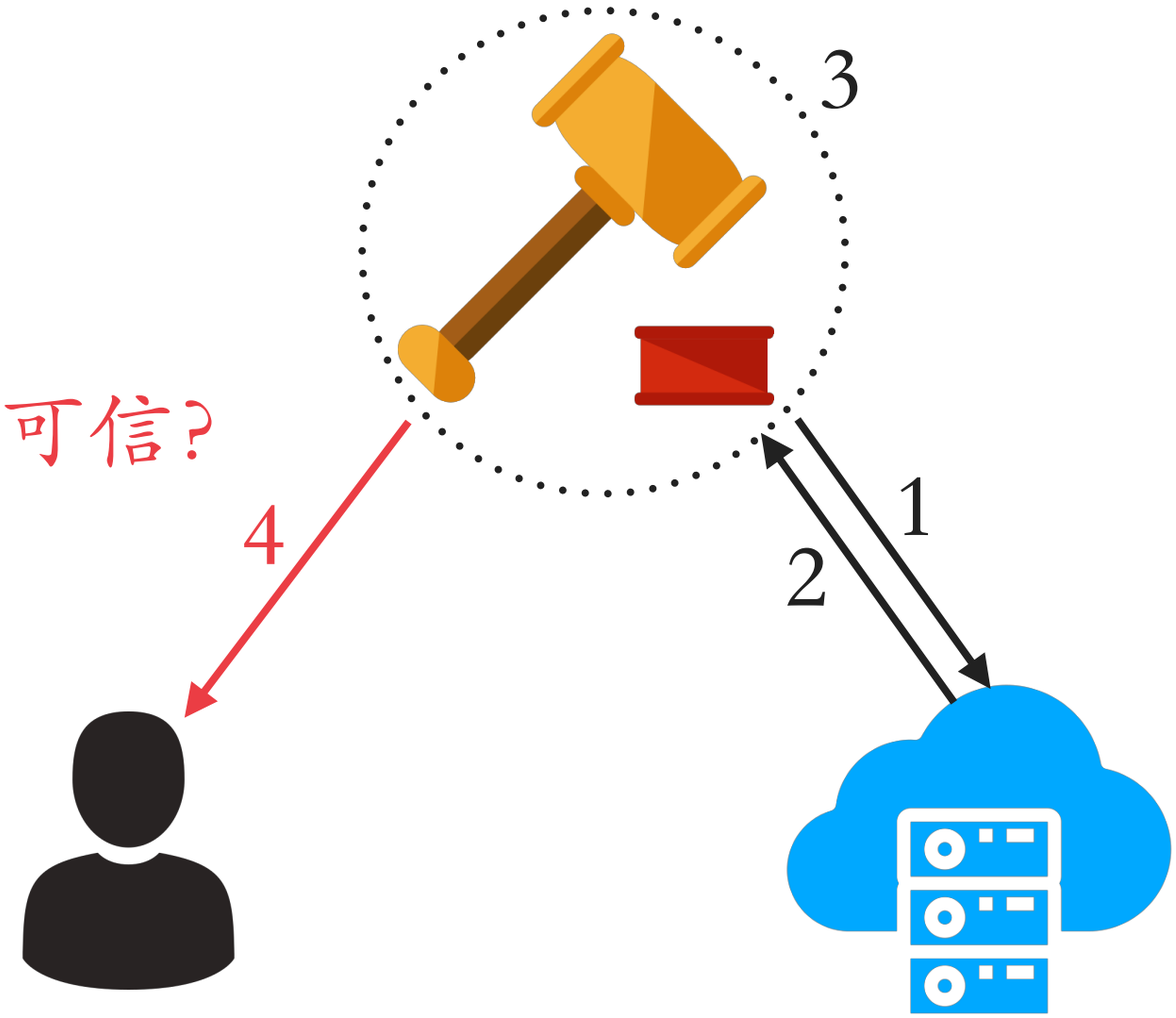


- 验证过程公正？

不一定！

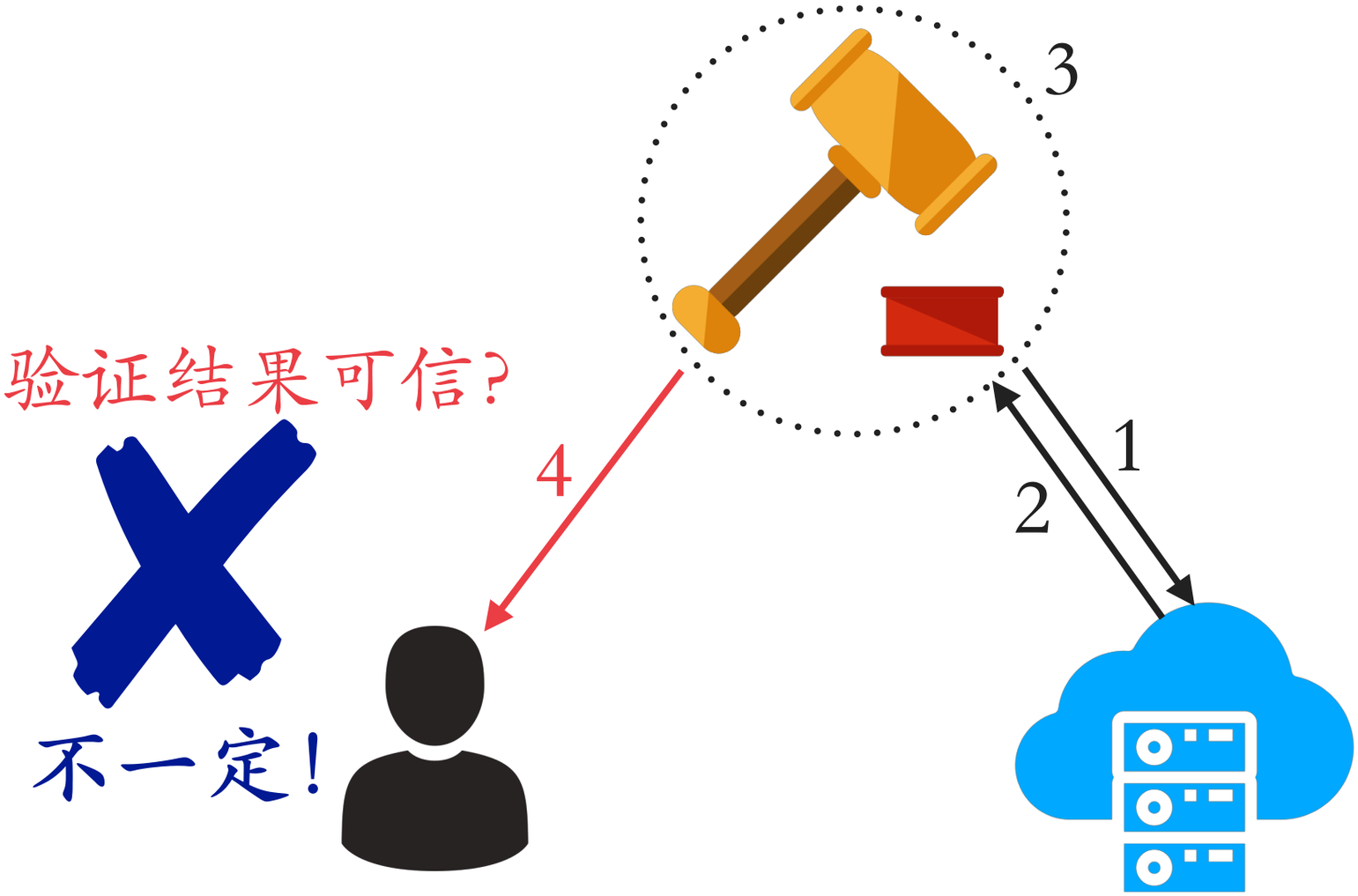
中心式审计模型存在的问题

- 验证结果可信?



中心式审计模型存在的问题

- 验证结果可信?



中心式审计模型的弊端

挑战：无法确保随机性

结果：不够可信（不可抵赖性）

威胁：云可以与审计者合谋

实际上，

“可信第三方”审计者是一个缓兵之计，

因为拿不出好的方法实现真正可信的不可抵赖性。

区块链技术提供了一个好的解决思路

区块链技术提供了一个好的解决思路

用算法建立信任

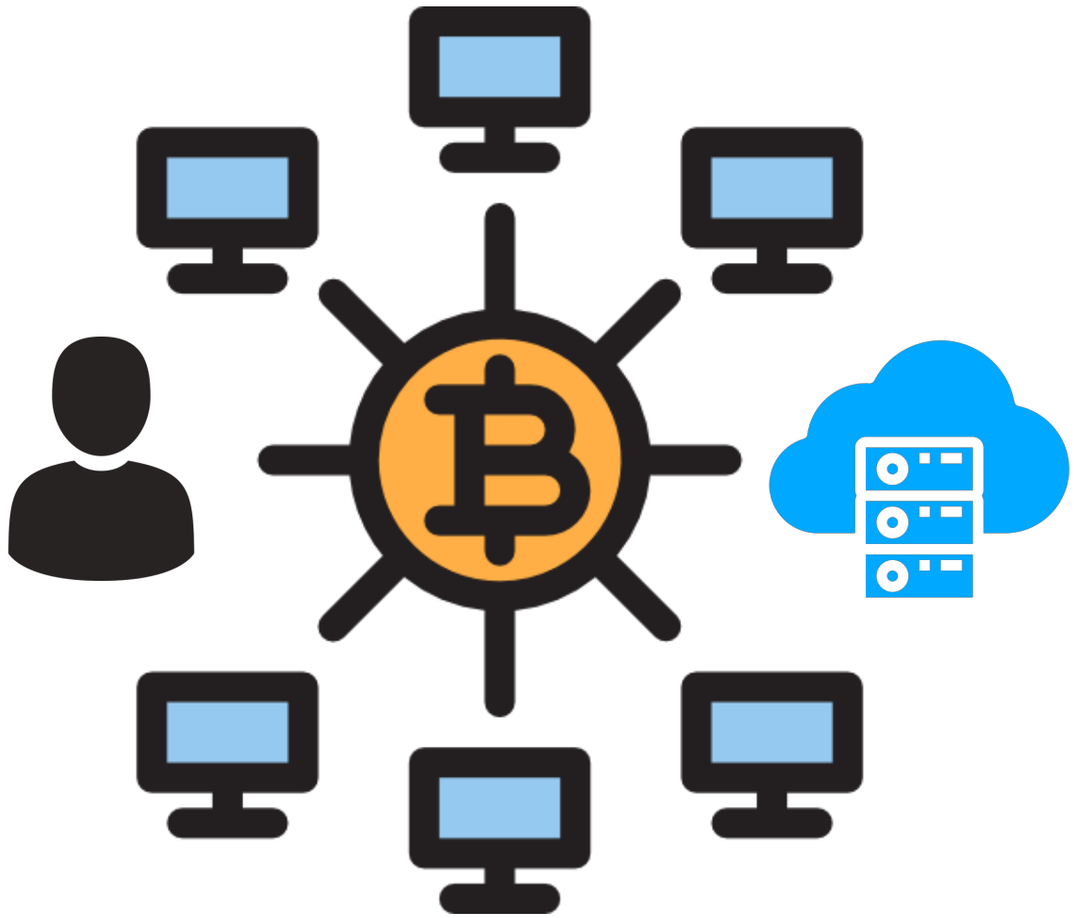
云存储的分布式数据完整性审计

借助区块链技术实现不可抵赖性

基于区块链的分布式审计模型(1/4)

NBJL

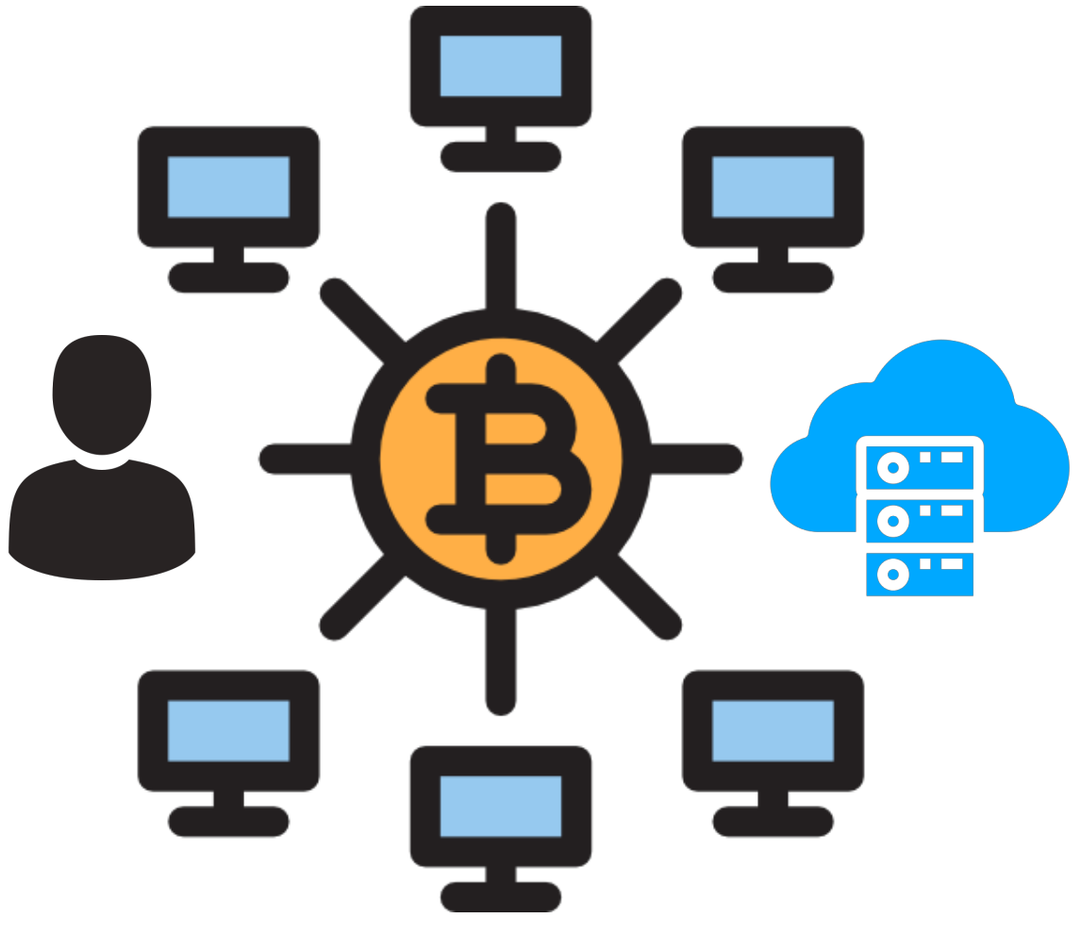
- 两个参与者
  - 用户
  - 云



基于区块链的分布式审计模型(2/4)

NBJL

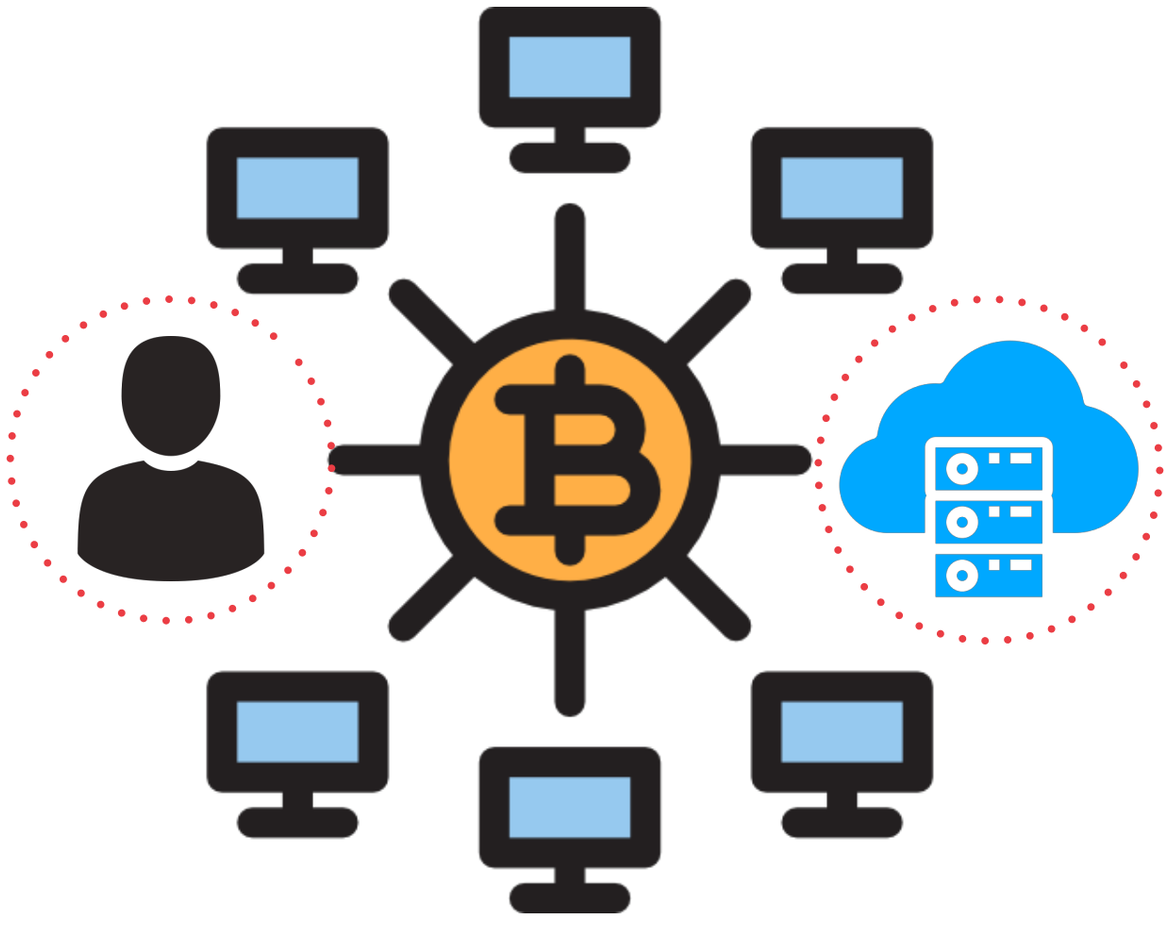
1. 审计者发起挑战
2. 云进行响应
3. 审计者进行验证
4. 结果告知用户



基于区块链的分布式审计模型(3/4)

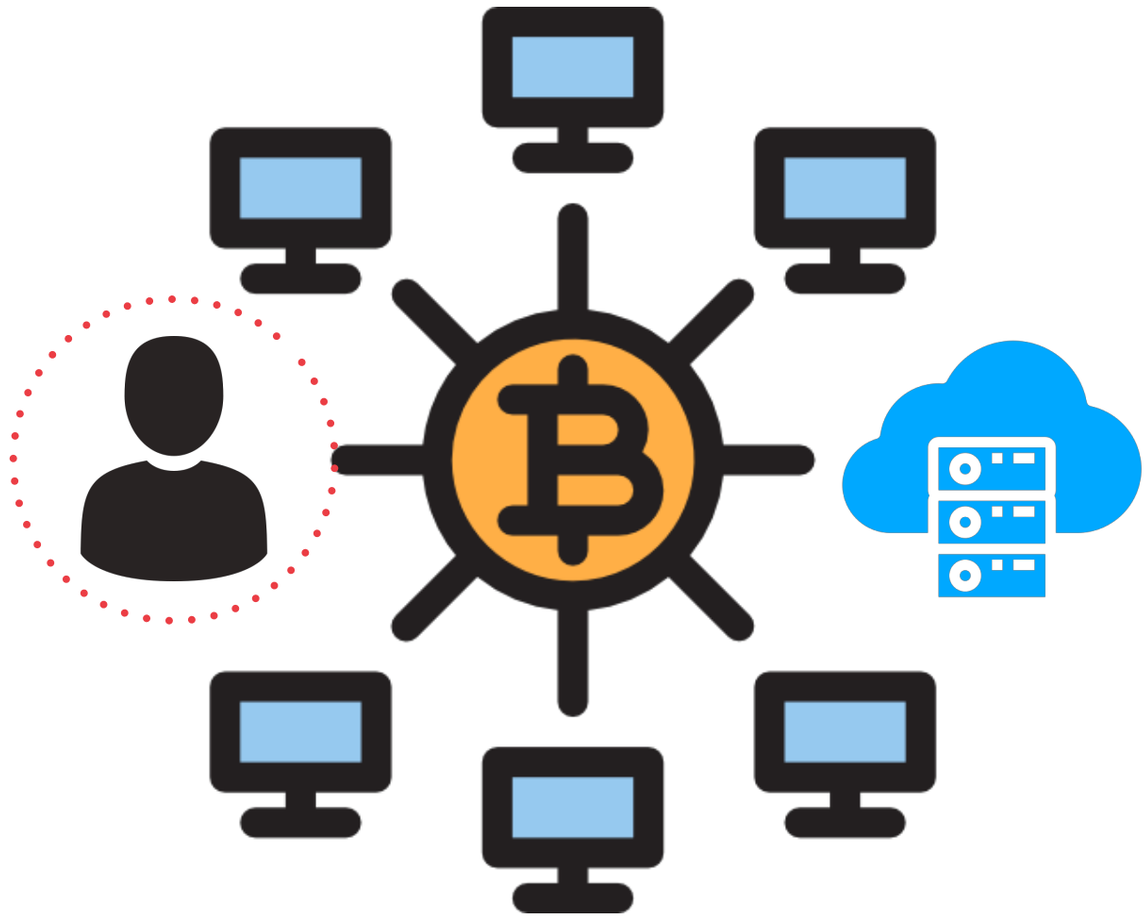
NBJL

1. 审计者发起挑战
2. 云进行响应
3. 审计者进行验证
4. 结果告知用户



基于区块链的分布式审计模型(4/4)

1. 审计者发起挑战
2. 云进行响应
3. 审计者进行验证
4. 结果告知用户

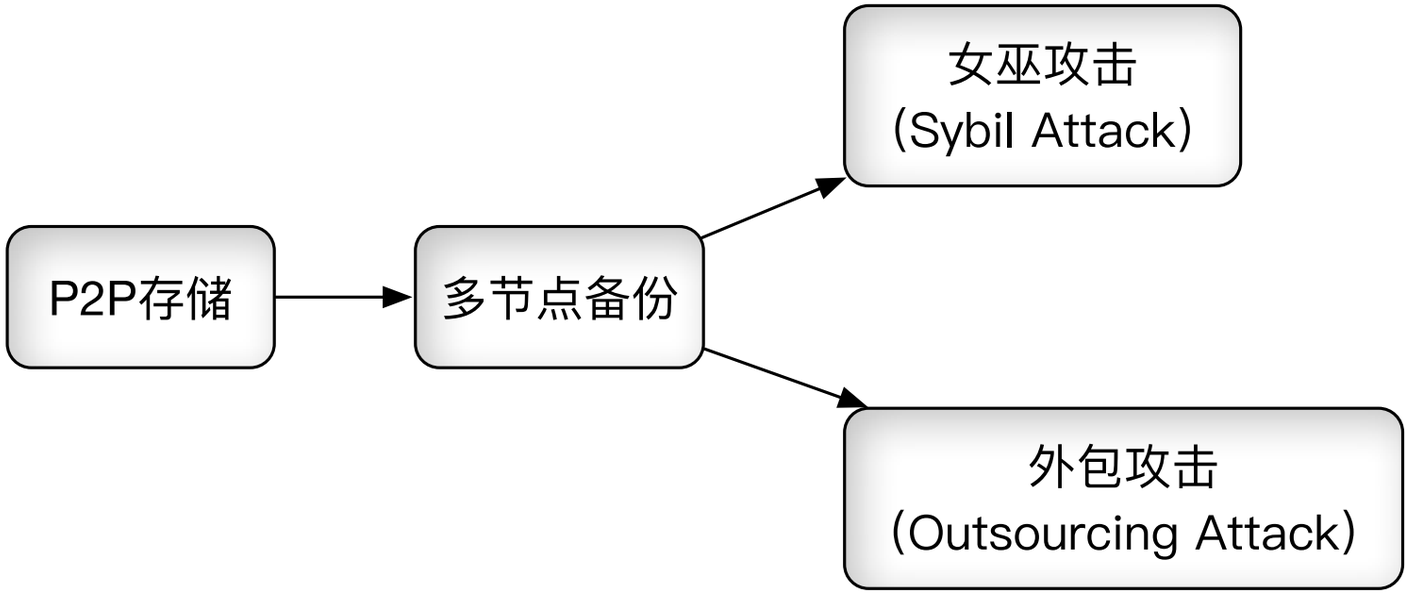


分布式存储网络对比

名称	挑战者	验证者
Filecoin	存储节点	所有节点
Storj	数据所有者	数据所有者
Permacoin	存储节点	所有节点
Retricoin	存储节点	所有节点

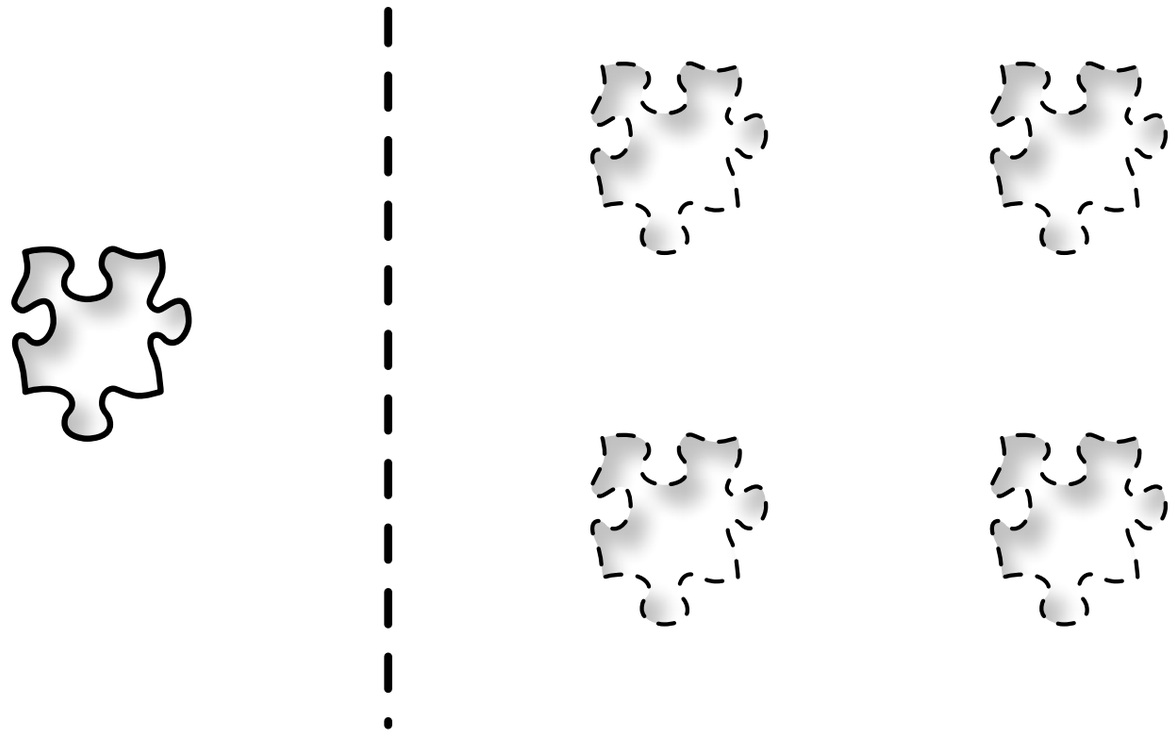
FILECOIN

- Proof of Replication



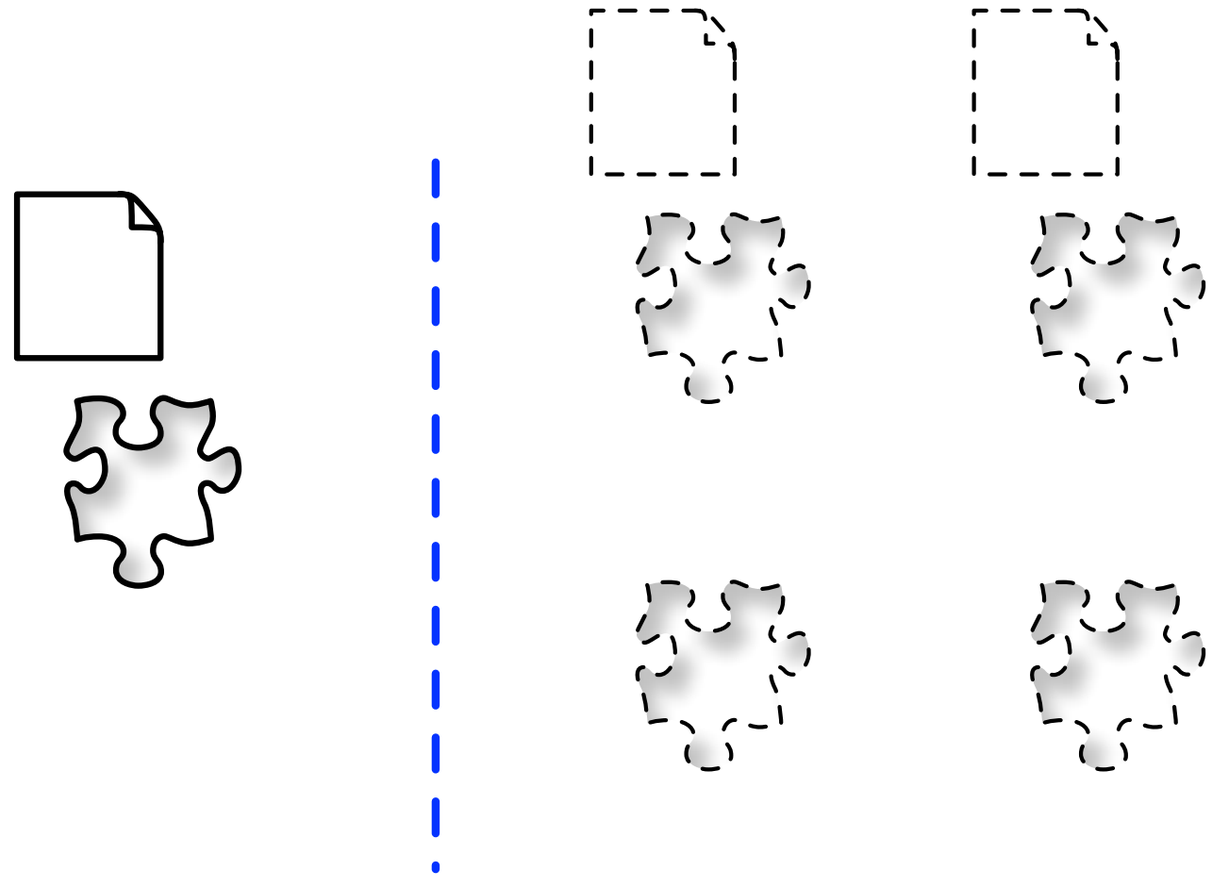
FILECOIN

- 女巫攻击



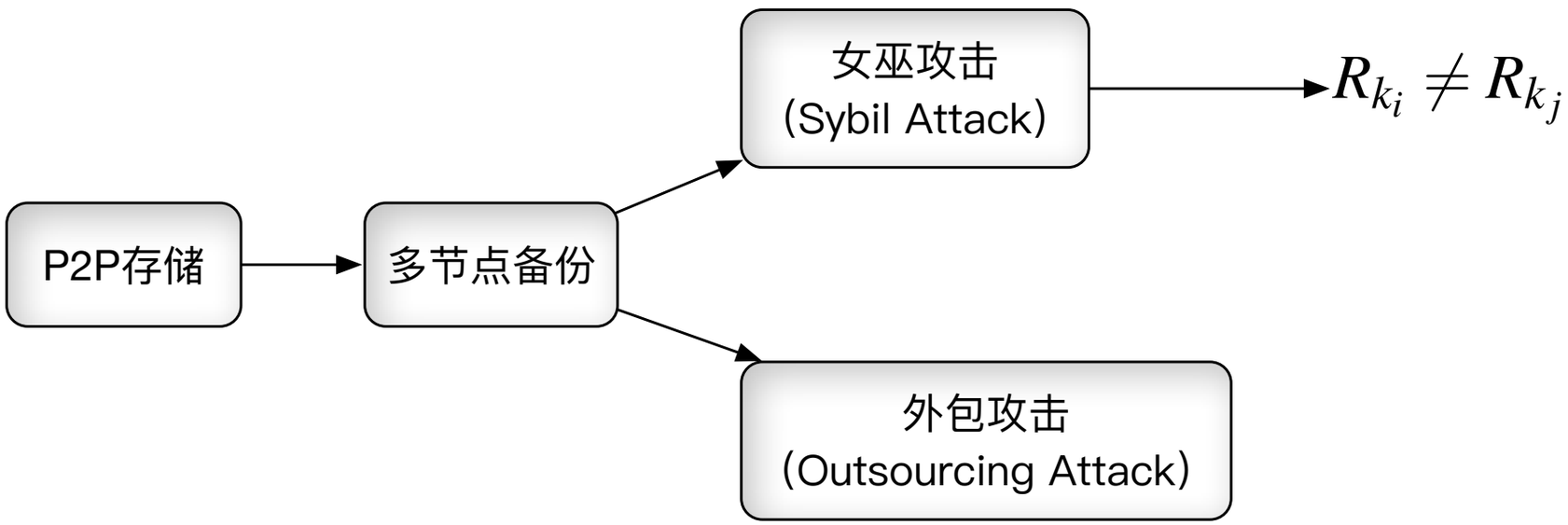
FILECOIN

- 女巫攻击



FILECOIN

- 解决女巫攻击的对策

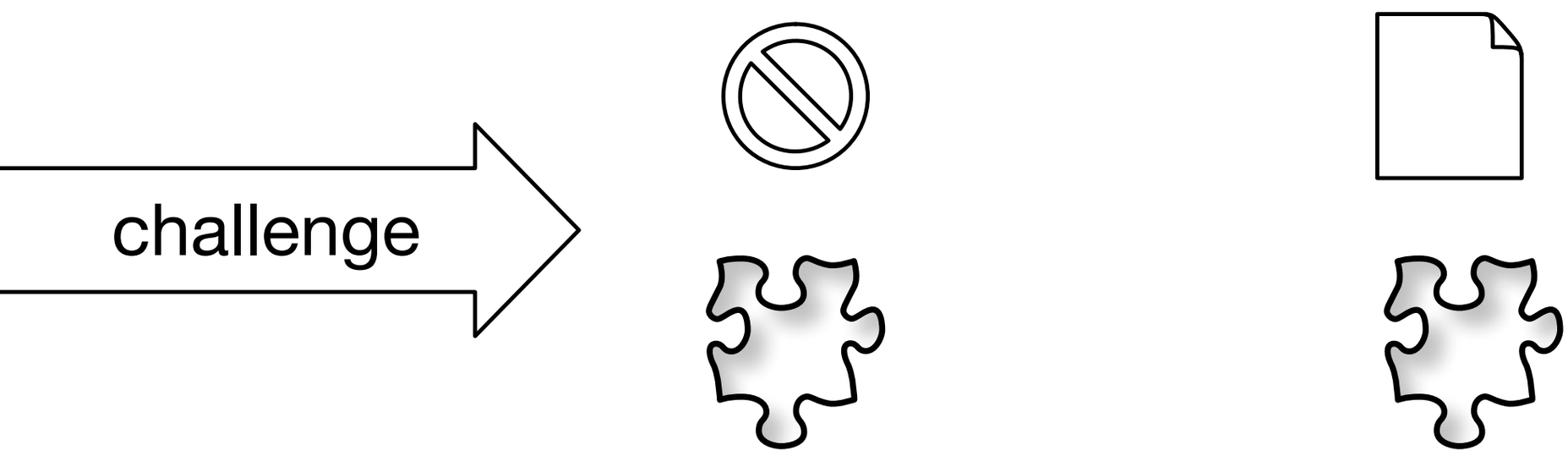


FILECOIN

- 外包攻击

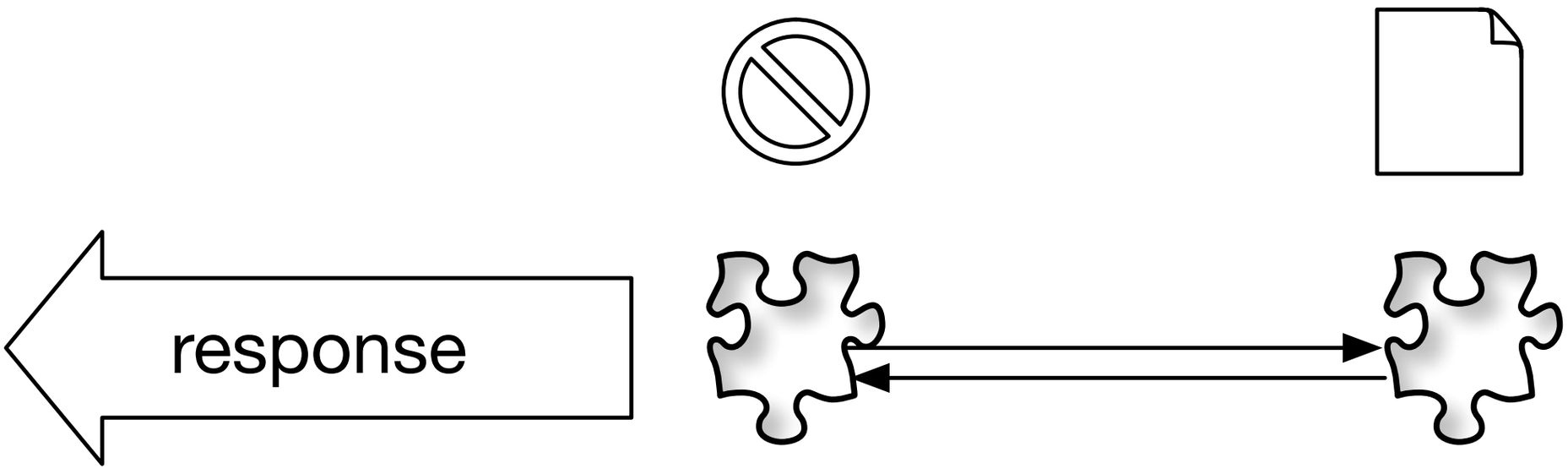
FILECOIN

- 外包攻击



FILECOIN

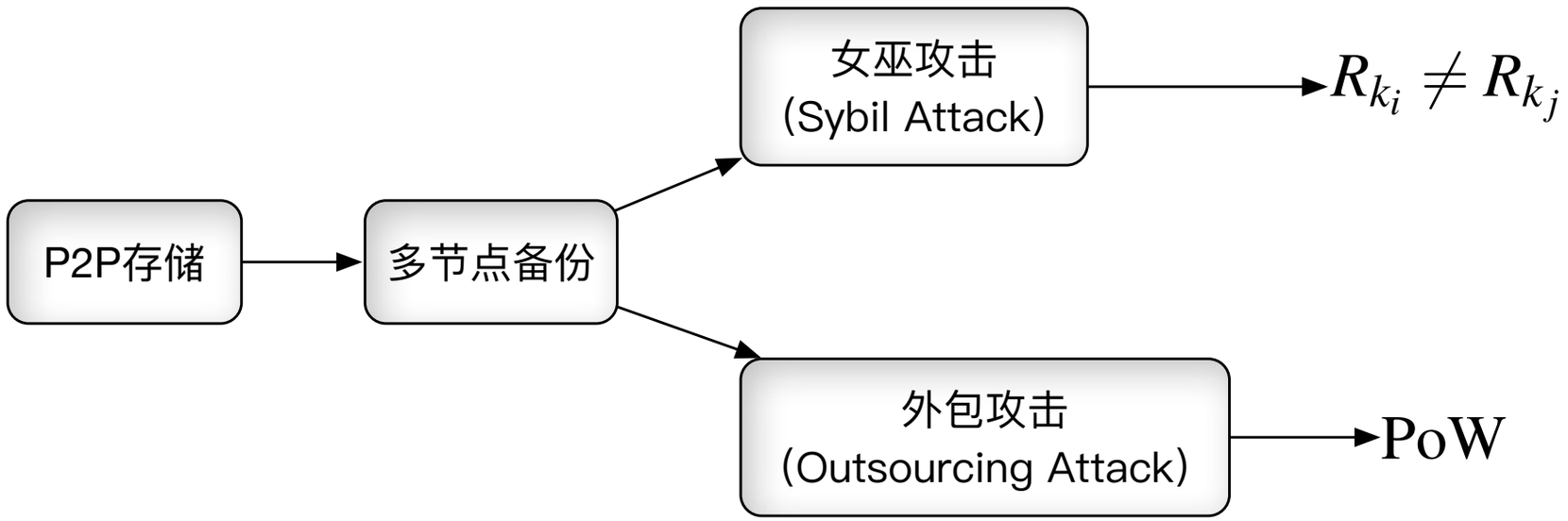
- 外包攻击



FILECOIN

- 解决外包攻击的对策

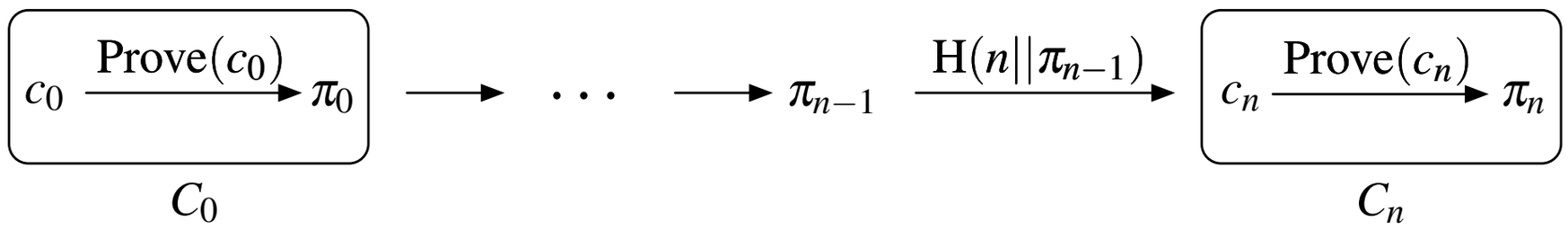
令证明过程相对耗时，证明者必须在规定时间内给出应答。



FILECOIN

- Proof of Spacetime

存储节点主动向用户证明自己在某段时间内完好地保存了一份副本，证明期间用户可以离线并后期查看记录。



PERMACOIN

- 简易版本

- **Setup.** The dealer computes and publishes the digest of the entire dataset, consisting of n segments
  A participant with public key pk chooses a subset S_{pk} of segments to store:

$$\forall i \in [\ell] : \text{ let } \mathbf{u}[i] := H_0(\mathsf{pk}||i) \mod n, \qquad S_{\mathsf{pk}} := \{\mathbf{u}[i]\}_{i\in[\ell]}$$

  where ℓ is the number of segments stored by each participant. The participant stores $\{(\mathbf{F}[j], \pi_j)|j \in S_{\mathsf{pk}}\}$, where π_j is the Merkle proof for the corresponding segment $\mathbf{F}[j]$.

- **Scratch-off.** Every scratch-off attempt is seeded by a random string s chosen by the user. Let puz denote a publicly known, epoch-dependent, and non-precomputable puzzle ID. A participant computes k random challenges from its stored subset S_{pk}:

$$\forall i = 1, 2, \ldots, k : \quad r_i := \mathbf{u}[H(\mathsf{puz}||\mathsf{pk}||i||s) \mod \ell] \tag{1}$$

  The ticket is defined as:

$$\text{ticket} := (\mathsf{pk}, s, \{\mathbf{F}[r_i], \pi_i\}_{i=1,2,\ldots,k})$$

  where π_i is the Merkle proof for the r_i-th segment $\mathbf{F}[r_i]$.

- **Verify.** The Verifier is assumed to hold the digest of F. Given a ticket $:= (\mathsf{pk}, s, \{\mathbf{F}[r_i], \pi_{r_i}\}_{i=1,2,\ldots,k})$ verification first computes the challenged indices using Equation (1), based on pk and s, and computing elements of $\mathbf{u}$ as necessary. Then verify that all challenged segments carry a valid Merkle proof.

PERMACOIN

- 防外包版本

- **Setup.** Let $r > 1$ denote a constant. Suppose that the original dataset F_0 contains f segments.

  First, apply a maximum-distance-separable code and encode the dataset F_0 into F containing $n = rf$ segments, such that any f segments of F suffice to reconstruct F_0. Then, proceed with the **Setup** algorithm of Figure 1.

- **Scratch-off.** For a scratch-off attempt seeded by an arbitrary string s chosen by the user, compute the following:

$$\begin{aligned}
&\sigma_0 := 0 \\
&r_1 := \mathbf{u}[H(\mathsf{puz}||\mathsf{pk}||s) \mod \ell] \\
&\text{For } i = 1, 2, \ldots, k: \\
&\quad h_i = H(\mathsf{puz}||\mathsf{pk}||\sigma_{i-1}||\mathbf{F}[r_i]) \quad (*) \\
&\quad \sigma_i := \mathsf{sign}_{\mathsf{sk}}(h_i) \\
&\quad r_{i+1} := H(\mathsf{puz}||\mathsf{pk}||\sigma_i) \mod \ell
\end{aligned}$$

  The ticket is defined as:

$$\mathsf{ticket} := (\mathsf{pk}, s, \{\mathbf{F}[r_i], \sigma_i, \pi_{r_i}\}_{\forall i=1,2,\ldots,k})$$

  where π_{r_i} is the Merkle proof of $\mathbf{F}[r_i]$.

- **Verify.** Given $\mathsf{ticket} := (\mathsf{pk}, s, \{\mathbf{F}[r_i], \sigma_i, \pi_{r_i}\}_{\forall i=1,2,\ldots,k})$, verification is essentially a replay of the scratch-off, where the signing is replaced with signature verification. This way, a verifier can check whether all iterations of the scratch-off were performed correctly.

PERMACOIN

- 最终版本

- KeyGen(y, ℓ): Let L denote the number of Merkle leaves. Pick $\{\sigma_1, \ldots, \sigma_L\}$ at random from $\{0,1\}^{O(\lambda)}$. Construct a Merkle tree on top of the L leaves $\{\sigma_1, \ldots, \sigma_L\}$, and let digest denote the root's digest.

  The secret key is sk $:= \{\sigma_1, \ldots, \sigma_L\}$, and the public key is pk $:=$ digest. The signer and verifier's initial states are $\Omega_s = \Omega_v := \emptyset$ (denoting the set of leaves revealed so far).

- Sign$(\mathsf{sk}, \Omega_s, m)$: Compute $H(m)$ where H is a hash function modelled as a random oracle. Use the value $H(m)$ to select a set $I :=$ sk $- \Omega_s$ of q unrevealed leaves. The signature is

$$\sigma := \{(\sigma_i, \pi_i)\}_{i \in I} \text{ in sorted order of } i$$

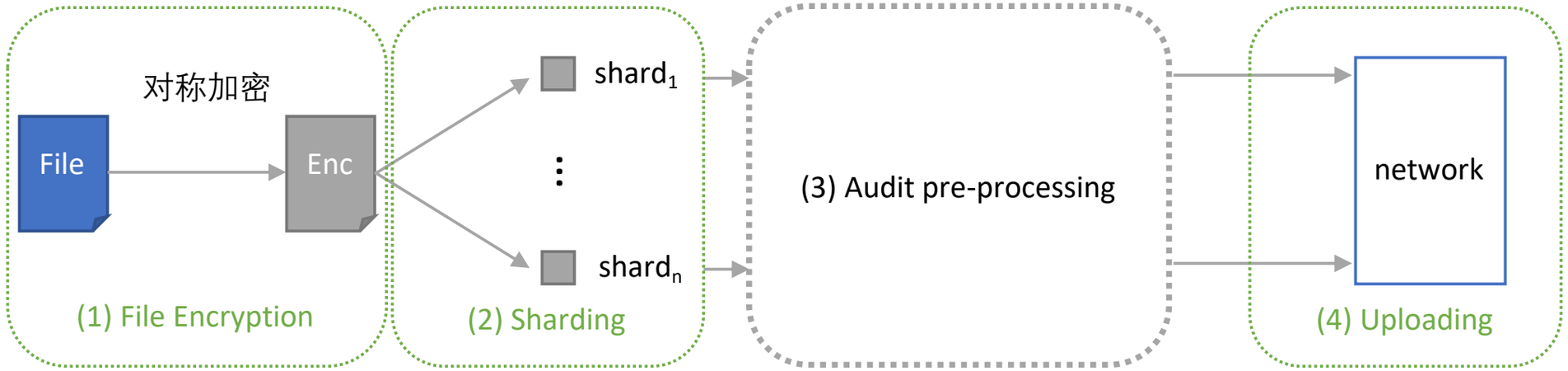
  where π_i is the Merkle proof for the i-th leaf. Update signer's state $\Omega_s := \Omega_s \cup \sigma$.

- Verify$(\mathsf{pk}, \Omega_v, m, \sigma)$: Use $H(m)$ to select an unused set I of leaves, of size q. Parse $\sigma := \{(\sigma_i, \pi_i)\}_{i \in I}$ (in sorted order of i). Verify that each π_i is a correct Merkle proof for the i-th leaf node σ_i where $i \in I$. Output 1 if all verifications pass. Otherwise output 0.

  Finally, update verifier's state $\Omega_v := \Omega_v \cup \sigma$.

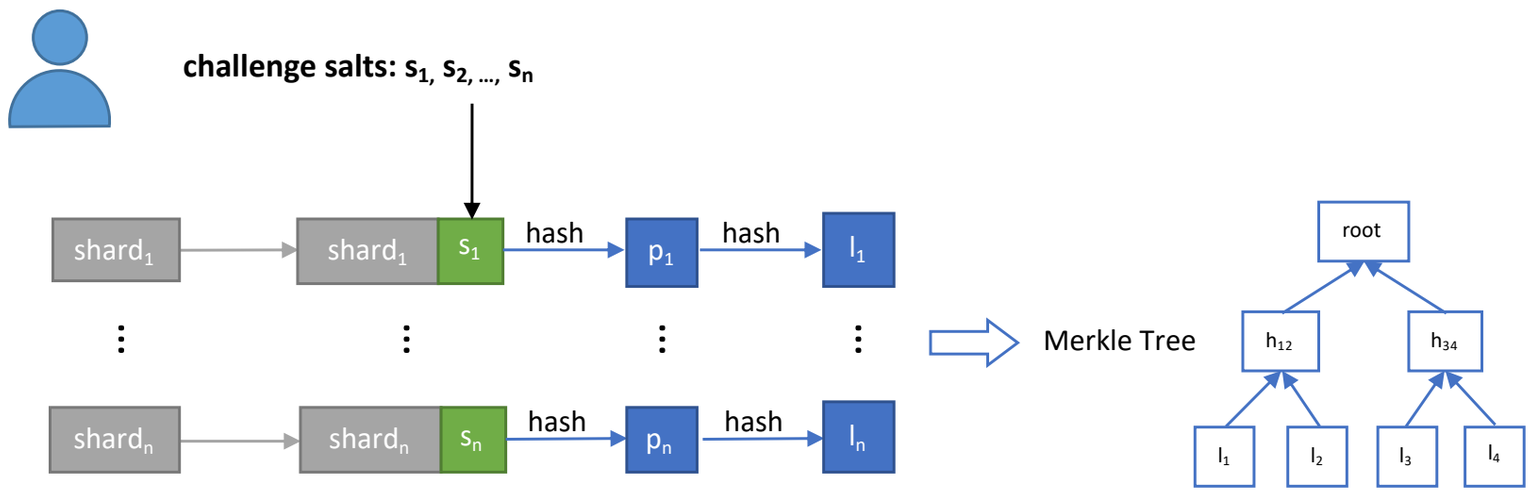
STORJ

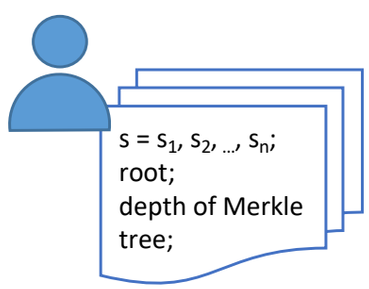
- 上传流程

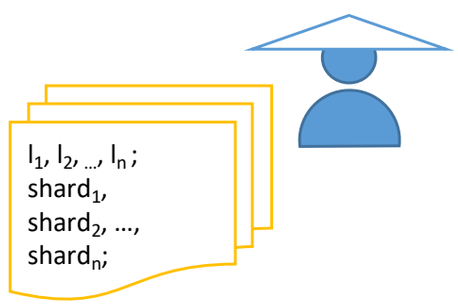

对称加密
File
Enc
shard1
shardn
(1) File Encryption
(2) Sharding
(3) Audit pre-processing
network
(4) Uploading

STORJ

- 存在性证明


challenge salts: $s_1, s_2, \ldots, s_n$
$shard_1$
$shard_1$
s_1
hash
p_1
hash
l_1
$shard_n$
$shard_n$
s_n
hash
p_n
hash
l_n
Merkle Tree
root
h_{12}
h_{34}
l_1
l_2
l_3
l_4


s = $s_1, s_2, \ldots, s_n$;
root;
depth of Merkle tree;


$l_1, l_2, \ldots, l_n$;
$shard_1$,
$shard_2$, …,
$shard_n$;